robotics

MDPI

Review

# A Systematic Literature Review of DDS Middleware in Robotic Systems

**Muhammad Liman Gambo [1], Abubakar Danasabe [1], Basem Almadani [1,2], Farouq Aliyu [1,2,*], Abdulrahman Aliyu [3] and Esam Al-Nahari [2]**

1 Computer Engineering Department, King Fahd University of Petroleum & Minerals, Dhahran 31261, Saudi Arabia; g202320790@kfupm.edu.sa (M.L.G.); g202319370@kfupm.edu.sa (A.D.); mbasem@kfupm.edu.sa (B.A.)
2 Center of Excellence in Development of Nonprofit Organizations, King Fahd University of Petroleum & Minerals, Dhahran 31261, Saudi Arabia; alnahari@kfupm.edu.sa
3 Applied Research Center for Metrology, Standards and Testing, King Fahd University of Petroleum & Minerals, Dhahran 31261, Saudi Arabia; abdulrahman.aliyu@kfupm.edu.sa
* Correspondence: farouq.muhammad@kfupm.edu.sa; Tel.: +966-13-860-5129

**Abstract:** The increasing demand for automation has led to the complexity of the design and operation of robotic systems. This paper presents a systematic literature review (SLR) focused on the applications and challenges of Data Distribution Service (DDS)-based middleware in robotics from 2006 to 2024. We explore the pivotal role of DDS in facilitating efficient communication across heterogeneous robotic systems, enabling seamless integration of actuators, sensors, and computational elements. Our review identifies key applications of DDS in various robotic domains, including multi-robot coordination, real-time data processing, and cloud–edge–end fusion architectures, which collectively enhance the performance and scalability of robotic operations. Furthermore, we identify several challenges associated with implementing DDS in robotic systems, such as security vulnerabilities, performance and scalability requirements, and the complexities of real-time data transmission. By analyzing recent advancements and case studies, we provide insights into the potential of DDS to overcome these challenges while ensuring robust and reliable communication in dynamic environments. This paper aims to contribute to the transformative impact of DDS-based middleware in robotics, offering a comprehensive overview of its benefits, applications, and security implications. Our findings underscore the necessity for continued research and development in this area, paving the way for more resilient and intelligent robotic systems that operate effectively in real-world scenarios. This review not only fills existing gaps in the literature but also serves as a foundational resource for researchers and practitioners seeking to leverage DDS in the design and implementation of next-generation robotic solutions.

**Keywords:** data distribution service; DDS; internet of things; middleware; real-time operating system; robotics; ROS 2; SDG9

check for updates

Academic Editor: Genci Capi

Received: 19 March 2025
Revised: 29 April 2025
Accepted: 8 May 2025
Published: 14 May 2025

**Citation:** Gambo, M.L.; Danasabe, A.; Almadani, B.; Aliyu, F.; Aliyu, A.; Al-Nahari, E. A Systematic Literature Review of DDS Middleware in Robotic Systems. *Robotics* **2025**, *14*, 63. https://doi.org/10.3390/robotics14050063

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

## 1. Introduction

Over the past few decades, robotics has undergone tremendous changes, with far-reaching consequences for fields as diverse as healthcare, industrial manufacturing, and autonomous systems. The development of robotic systems has been further explored by continuous advances in machine learning and artificial intelligence (AI), allowing robotic systems to execute complex tasks with greater autonomy and precision. Robotics

is a computing paradigm focusing on work process design and coordination through programming [1]. It needs to resolve three main issues [1]: First, building a physical body that efficiently uses energy to complete predetermined tasks; second, making sure robots can accomplish a goal within their environment or conclude it cannot achieve the goal (due to lack of resources); and third, creating a program or design that ensures accomplishing goals within a specific task domain.

Additionally, ref. [2] defines robotics as a diverse field that integrates aspects of electrical engineering, computer science, and mechanical engineering to create machines that can operate with partial or complete autonomy. In addition, refs. [3,4] describes robotics as the study of automated devices, or robots, that combine mechanical systems, sensors, control algorithms, and algorithms to carry out tasks independently or under human supervision. Both definitions highlight how hardware and intelligent control systems work together to perform various tasks. These explanations emphasize how crucial robotics is to the ability of a machine to interact with and adjust to its environment, either on its own or with assistance from a human. Garcia et al. [5] reviewed the development of robotics research over time. They looked at how industrial robotics that automate hazardous manufacturing tasks moved to make way for service robotics, which could meet a greater variety of human needs. They also highlighted advances in intelligent control methods and the expanding need for robots across different industries, including the medical and rehabilitation domains. Also, Ben-Ari and Mordechai [6] presented a thorough examination of robotics with a particular emphasis on education and functionality. Their work emphasizes how programmable and capable robots are at handling challenging tasks.

The development of robotic systems requires many heterogeneous components, such as actuators, control units, sensors, and computational elements, to combine and perform autonomous or semi-autonomous tasks. The increasing complexity of robots presents significant challenges in synchronizing, controlling, and communicating among these diverse components. For modern robots to operate successfully, coordination between the hardware and software layers is vital. These challenges motivate middleware creation, a mediator that facilitates communication, improves interoperability, and optimizes resource allocation among robotic system components. Baumann and Martinoli [7] tackled the increasing diversity and complexity challenges of robotic systems using a modular framework and middleware solutions such as the Robotic Operating System (ROS). Middleware frameworks like the ROS offer portability and modularity, allowing developers a seamless transition from simulation environments to real-world robots. In addition, middleware solutions enhance system interoperability and enable robots to perform better in various real-world applications by removing the complexity of hardware and software layers.

*Contributions*

This systematic literature review (SLR) aims to enhance the understanding of DDS-based middleware and its applications in robotic systems. Our review addresses the gaps in existing research and offers comprehensive insights into the potential applications and challenges of DDS-based middleware in robotics. The contributions of this paper are as follows:

1. We provide an in-depth examination of how DDS middleware enhances communication, interoperability, and coordination in robotic systems, focusing on real-time performance and scalability across diverse application domains.
2. We present a structured taxonomy of DDS-based middleware applications in robotics, organizing the literature by communication paradigms, system architectures, and functional domains to support future research and development.

3. We investigate key challenges in implementing DDS in robotic systems, including middleware integration, scalability, configuration complexity, and performance bottlenecks in real-time environments.
4. We also discuss the security and privacy implications of applying DDS in robotics, highlighting current vulnerabilities and evaluating the tradeoffs in implementing secure communication mechanisms. This topic is addressed separately from key challenges due to the depth and volume of relevant findings in the reviewed literature.

The remaining sections of the paper are as follows: Section 2 presents the SLR methodology, including the research questions (RQs), the information sources, the search strategy, and the article screening and selection process. Section 3 discusses the theoretical background of middleware and DDS middleware, including its architecture, quality of service, and security features. Sections 4–6 answer the research questions RQ1, RQ2, and RQ3 presented in the methodology section (i.e., Section 2), respectively, discussing each in detail. Finally, Section 7 consists of the conclusion and future direction.

## 2. Methodology

Robotic integration with DDS middleware significantly improves real-time communication, interoperability, and scalability across diverse applications. Its relevance spans diverse critical areas, enabling efficient data exchange, enhancing performance, and addressing the complex demands of modern robotic systems. This systematic literature review aims to answer the following research questions:

- RQ1: What are the applications of DDS-based middleware in robotics?
- RQ2: What challenges come with DDS implementation in robotics?
- RQ3: What security and privacy implications does DDS-based middleware have in robotics?

This study used the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) methodology to answer these questions, guaranteeing a comprehensive and open selection of relevant articles [8]. To ensure the accuracy and reliability of the research results, PRISMA provides a 27-item checklist to writers, editors, readers, and peer reviewers. Figure 1 shows the process flowchart for selecting and evaluating the collected articles. The process follows the PRISMA criteria. Also, the study covers the literature published between 2006 and 2024, ensuring a thorough analysis of past and emerging trends in DDS-based robotics middleware. The following subsections describe the remaining components of the methodology.

### *2.1. Information Sources*

This study used two popularly known databases, Scopus and Web of Science (WoS), to identify relevant publications for the study. These databases are known for their extensive range of articles and credibility in hosting high-quality publications. They provide a replicable and structured mechanism for searching and identifying applicable articles. They also have broad coverage and credibility in indexing the peer-reviewed literature in multiple disciplines. Moreover, we limited the database to only two to reduce duplicates in the identification process.

### *2.2. Search Strategy*

We identified keywords relevant to the research questions to help us search the two databases and identify and retrieve publications related to this study. We ensured the search terms convey the same information, since the two databases are dissimilar. The search terms looked for publications containing “data distribution service” or “dds” in addition to the words “middleware” and “robot” in their title, abstract, or keywords. These

keywords were selected because this work focuses primarily on the applications of DDS middleware and the challenges associated with its implementation in robotics.

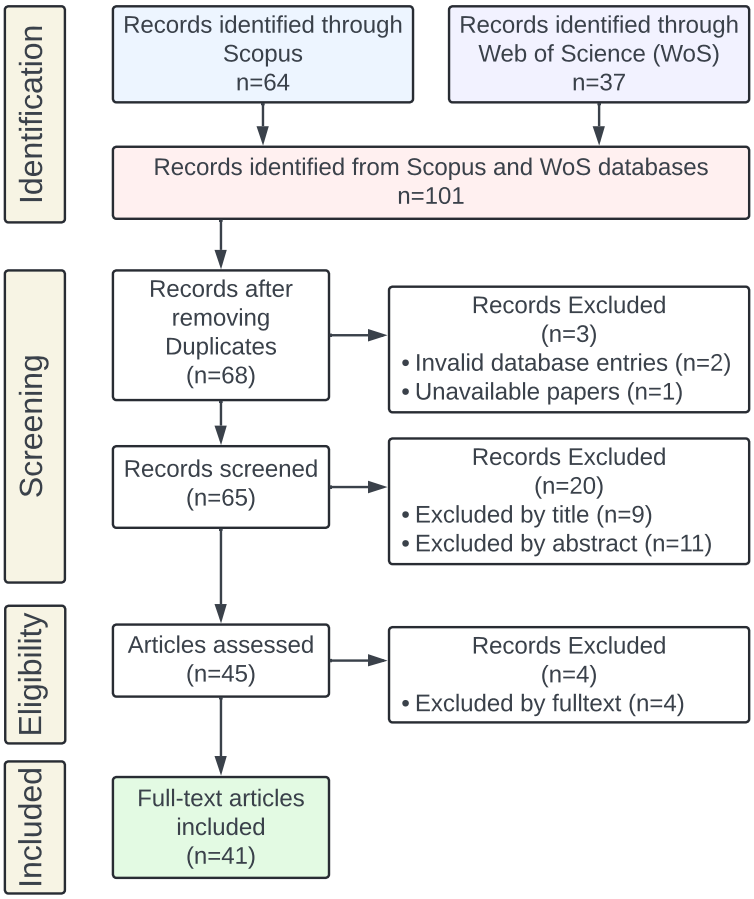



**Figure 1.** Article identification and selection process using PRISMA framework.

### 2.3. Screening and Selection Process

In addition, Figure 1 shows that the article selection strategy used in this SLR follows a four-stage process. These stages are Identification, Screening, Eligibility, and Inclusion. The Identification phase involves searching the Scopus and WoS databases in the process mentioned in the Search Strategy section (Section 2.2). The search in Scopus yielded 64 publications, while the WoS database returned 37 relevant articles, totaling 101 papers. To ensure accuracy and avoid redundancy, we cross-checked the results from both databases to identify any overlapping entries. This comparison revealed that 33 publications are common to both Scopus and WoS. After carefully removing these duplicate records, 68 unique publications remained. These remaining studies formed the initial pool of literature for the subsequent screening and eligibility assessment stages.

The Screening and Eligibility stages evaluated the publications using the inclusion and exclusion criteria outlined in Tables 1 and 2. Each table has two columns. The first column shows the attributes under consideration, while the second describes the conditions for including or excluding the publication from the pool. A publication must meet all the inclusion criteria and should not have any of the exclusion attributes to be a member of the final review paper pool. As a result, 45 articles met the two criteria after being screened according to titles and abstracts. In the final stage of the process, we found that four papers were ineligible after reading the full text. Thus, 41 articles made the pool for the literature review.

**Table 1.** Table of attributes and descriptions.

| Attribute | Description |
|---|---|
| Focus | The papers should focus on DDS-based middleware implementation in robotics, addressing specific application areas, associated challenges, or potential security and privacy concerns. |
| Type | We considered all scientific publications, including peer-reviewed articles, conference papers, and review papers. |
| Relevance | The papers must utilize DDS-based middleware as a communication channel in robotics applications. |
| Recency | We included all published papers from 2006 to 2024 to capture historical developments and emerging trends. |

**Table 2.** Table of Exclusion Criteria.

| Attribute | Description |
|---|---|
| Duplicates | We excluded articles with substantial content overlap to maintain diversity and originality in the review. |
| Sources | Exclude all non-peer-reviewed materials, such as websites, blogs, and opinion pieces, to ensure the academic integrity of the review paper. |
| Completeness | We eliminated publications that were either incomplete, not accessible, or invalid. |
| Language | We excluded all papers published in languages other than English. |

### *2.4. Generative AI Usage*

To enhance the efficiency, consistency, and clarity of this SLR, we incorporated AI tools at multiple stages of the research process. Figure 2 illustrates the AI integration workflow used in this study. In the diagram, the white blocks represent human activities, while green-shaded blocks indicate tasks carried out by AI. Following the identification of the final pool of relevant papers (as described in the Screening and Selection Process section), we manually read and summarized each article. These summaries were then processed using Chatsonic AI tools [9], which extracted keywords from the text. The extracted keywords were filtered to retain only those relevant to the applications of DDS in robotics, forming the foundation of the taxonomy that guided the writing of the applications section (Section 4).

During the writing phase, ChatGPT (GPT-4-turbo) [9] and Grammarly AI (free version) [10] were used iteratively on each paragraph to ensure the final manuscript met academic writing standards. ChatGPT assessed their readability. Where necessary, it was employed to enhance coherence and flow. Then, Grammarly checked the paragraphs for grammar and spelling errors. Afterward, each AI-suggested modification was manually reviewed to verify that the original meaning of the paragraph was preserved. This AI-assisted methodology enabled a more structured and accelerated synthesis of complex literature while improving the clarity and quality of the final manuscript.

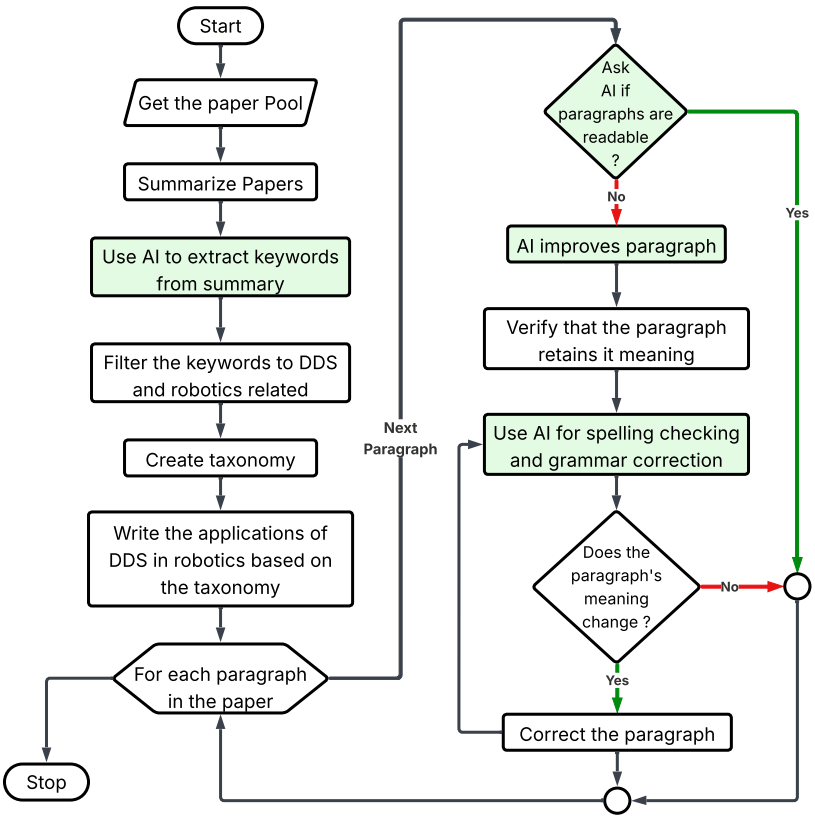



**Figure 2.** Flowchart for AI usage in this research.

## 3. Middleware Overview

This section presents an overview of middleware. It defines a middleware and its possible applications. Then, it focuses on DDS middleware, presenting its architecture, Quality of Service (QoS), and security features. This section aims to help the reader understand the remaining part of the paper.

### *3.1. Middleware*

As heterogeneous systems grow, their complexity and diversity increase. Thus, ensuring efficient system communication and function becomes an increasingly difficult task. Middleware solutions have emerged as crucial tools to address these challenges, enabling subsystems to interact effectively and ultimately improving performance, scalability, and flexibility. Many researchers and experts have attempted to define middleware. For example, Bishop and Karne define middleware as software that enables communication and interaction between applications, networks, hardware, and operating systems [11]. Almadani et al. [12] define it as a software layer that facilitates communication and data management between applications, operating systems, and hardware in distributed computing environments, simplifying system development and enhancing functionality while remaining transparent to end users.

Figure 3 shows a typical middleware architecture. It integrates various components, allowing developers to manage the complexity of distributed systems. Hence, Sadok et al. [13] define middleware as distributed software infrastructure to integrate, control, and communicate various components, such as physical devices, actuators, sensors, and applications. These definitions have a common objective: enabling smooth integration and communication between systems and components, guaranteeing interoperability, and effective operation in complex technological environments.

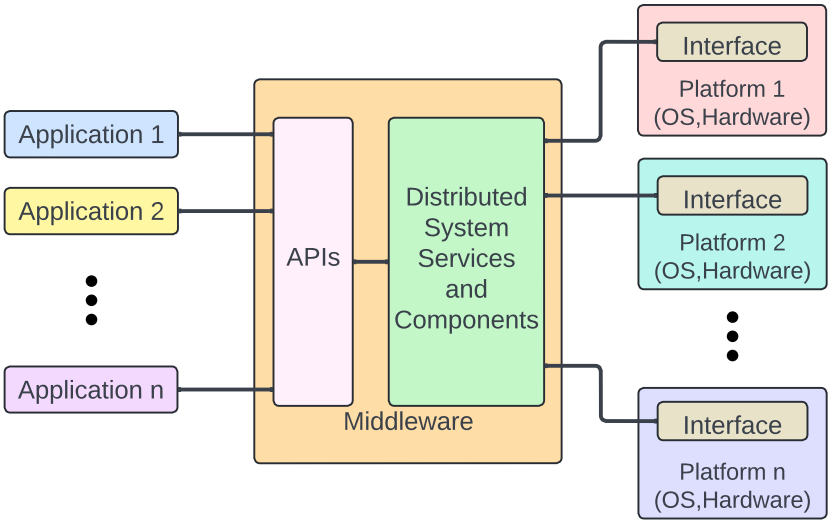



**Figure 3.** Top-level architecture for typical middleware.

### 3.2. Types of Middleware

Middleware can be categorized into four main types according to its features and applications, as illustrated in Figure 4. This classification provides a structured approach to understanding the different paradigms for facilitating communication and interoperability in distributed systems.

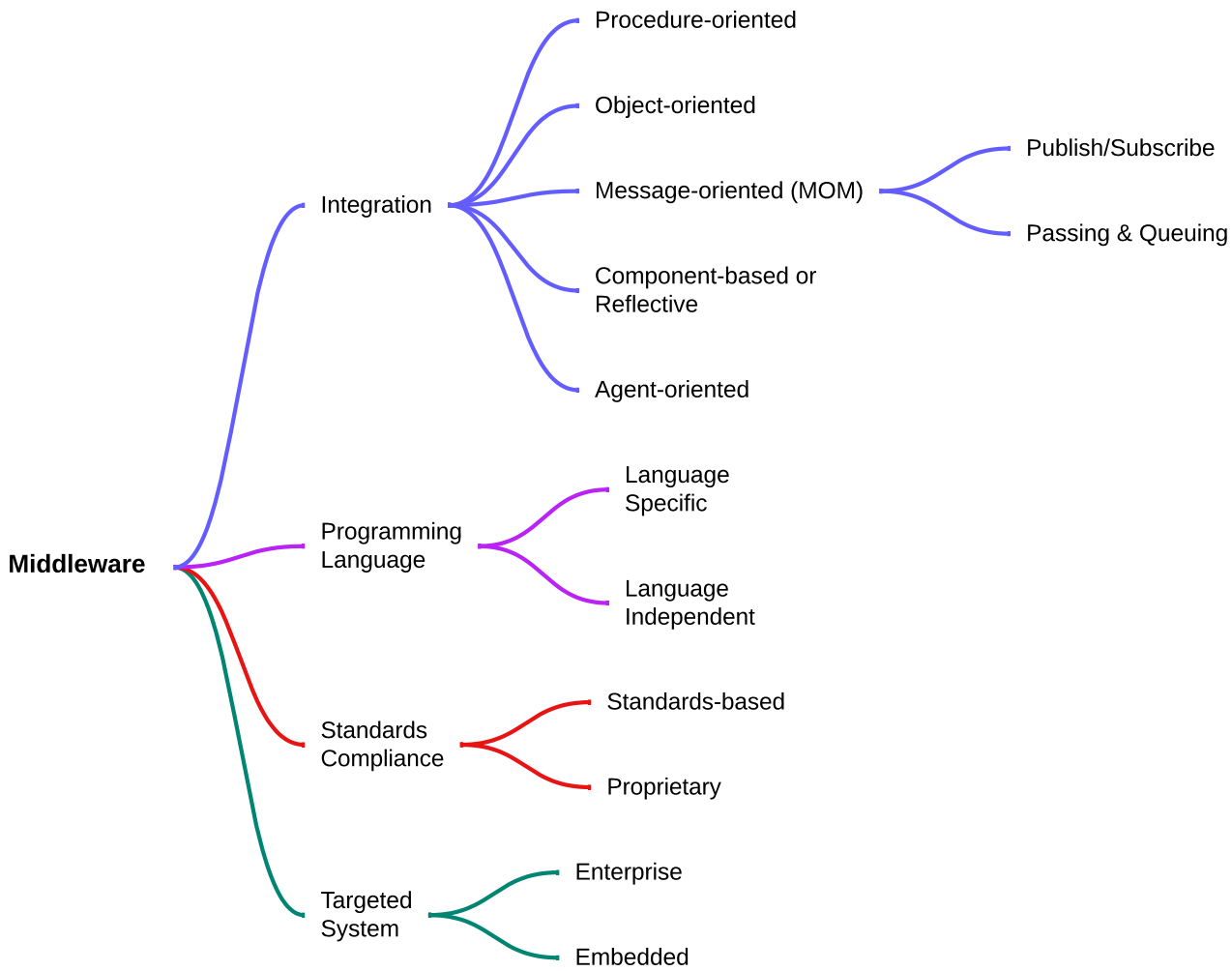



**Figure 4.** Middleware taxonomy.

The first category classifies middleware by integration type, which defines the underlying design principles governing how middleware enables interaction among heterogeneous and distributed systems. Bishop and Karne presented this category in [11]. Procedure-oriented middleware relies on Remote Procedure Calls (RPCs), where a client application invokes a procedure on a remote server as if it were a local function [14]. This approach uses client stubs and server skeletons to handle communication between applications [15]. Object-oriented middleware extends this concept by enabling interaction between distributed objects, offering various synchronization mechanisms to coordinate communication [15]. Message-oriented middleware (MOM) supports asynchronous communication by facilitating message exchanges and operates under two primary models: message passing/queueing, where messages are stored in a queue until retrieved by the receiving application, ensuring reliable delivery even in cases of temporary disconnection, and publish/subscribe, where messages are broadcasted to multiple subscribers based on predefined topics, enabling event-driven communication [16]. A component is a modular part of a software system that encapsulates a specific functionality or behavior [17]. A component-based middleware uses components as the core building blocks to create adaptable systems and manage communication between them, and a reflective middleware is characterized by its ability to support dynamic adaptation of the system software [18]. Thus, researchers call a component-based middleware a reflective middleware because it combines component-based programming with reflection to enable dynamic adaptation and reconfiguration of the middleware system [18]. Lastly, an agent is a software entity that can act autonomously to perform tasks on behalf of users or other agents [19]. Thus, an agent-oriented middleware is designed for distributed environments where autonomous agents operate while interacting with each other to achieve predefined objectives [11].

The second classification of middleware is based on language dependency [15]. Middleware can be language-specific, which means it is tightly coupled to a particular programming language, such as C++ or Modula-3. Although this approach can provide optimized performance within a specific development ecosystem, it limits interoperability across different programming languages. Examples of some language-specific middleware are the Java Message Service (JMS), which allows Java applications to create, send, receive, and read messages in a loosely coupled asynchronous manner [20], and the Windows Communication Foundation (WCF) middleware framework that supports various communication protocols, including SOAP, REST, and TCP, for distributed applications in the .NET ecosystem [21]. Conversely, language-independent middleware ensures compatibility between applications in different languages, typically through standardized protocols and interfaces such as the Common Object Request Broker Architecture (CORBA) [15].

Another important middleware classification is by standards compliance. These are standard-based and proprietary [15]. Standards-based middleware adheres to predefined specifications set by standardization bodies such as the Object Management Group (OMG), ensuring broad compatibility and interoperability across various systems and vendors. For example, RTI Connext [22] and FastDDS [23] are implementations of the data distribution service (DDS) standard [24], which is managed by the OMG. In contrast, proprietary middleware is vendor-specific and is often optimized for performance in controlled environments but may lack compatibility with other middleware solutions, thereby restricting interoperability. WCF is an example of a proprietary middleware developed by Microsoft. Although it supports industry standards such as SOAP and REST, it is tightly integrated with the .NET framework and is designed primarily for Windows-based applications. Microsoft has deprecated WCF in .NET Core and later versions, recommending gRPC or ASP.NET Core Web APIs as alternatives [25].

The final category classifies middleware based on the targeted system or the deployment environment. Middleware for enterprise systems supports large-scale business applications. It offers features like transaction management, scalability, and security. These systems require robust middleware to handle large volumes of data and complex interactions between distributed components. RTI Connext is a typical example of an enterprise middleware. Embedded middleware is for resource-constrained environments, such as Internet of Things (IoT) devices and real-time systems. These middleware solutions prioritize efficiency and lightweight execution to accommodate limited processing power, memory, and energy availability. TinyDDS is a notable embedded system middleware [26]. It is an excellent choice for low-power, real-time embedded systems that require efficient, scalable, and high-performance communication. Its lightweight nature makes it ideal for IoT, robotics, industrial automation, and safety-critical applications where conventional DDS implementations might be too heavy.

### 3.3. Middleware Applications

Middleware has various applications across different fields: Researchers used middleware in healthcare to enhance patient care and resolve heterogeneous medical technologies integration issues [27,28]. They also find application in wearable sensor networks, simplifying real-time monitoring of patient vital signs and allowing for prompt interventions and better patient outcomes. Ahn et al. [29] demonstrated the benefits of using component-based open middleware architecture in a distributed computing system for autonomous navigation systems. Mohamed et al. [30] used middleware to improve the performance of cooperative Unmanned Aerial Vehicle (UAV) systems, making it easier for UAVs to communicate and perform operational tasks. The authors used two types of middleware: intermiddleware, which links multiple UAVs and integrates them with other systems, and intramiddleware, which links devices within a UAV. Baumann and Martinoli used middleware to improve operational efficiency, interoperability, and communication of an industrial multi-robot navigation system [7].

### 3.4. Data Distribution Service (DDS) Middleware

The DDS is a middleware standard developed and maintained by the OMG, which was initially released in 2004 [31,32]. DDS was designed to address the increasing demand for a middleware framework that supports real-time data-centric communication in heterogeneous distributed environments [33,34]. It has gained widespread adoption due to its flexibility and comprehensive Quality of Service (QoS) policies, which enable precise control over reliability, latency, and data persistence [31]. Over time, DDS has evolved through various updates and extensions, encompassing both open-source and commercial implementations. Many application domains are adopting it, including robotics, industrial IoT, autonomous systems, aerospace, defense, healthcare, and smart grid technologies [35,36]. Table 3 lists some prominent DDS middleware solutions available.

**Table 3.** Prominent DDS middleware solutions.

| Ref. | Middleware Name | Company/Organization | Free and Open Source | Version | Release Year |
|---|---|---|---|---|---|
| [37] | RTI Connext DDS | Real-Time Innovations (RTI) | No | 7.4.0 | Oct. 2024 |
| [38] | OpenDDS | Object Computing, Inc. (OCI) | Yes | 3.31.0 | Jan. 2025 |
| [23] | Fast DDS | eProsima | Yes | 3.2.0 | Mar. 2025 |
| [39] | CoreDX DDS | Twin Oaks Computing | No | 5.0.0 | 2020 |
| [40] | Vortex OpenSplice | ADLINK Technology | Yes | 6.9.0 | Mar. 2021 |
| [41] | GurumDDS | GurumNetworks | No | 3.2.0 | - |
| [42] | Cyclone DDS | Eclipse Foundation (via ZettaScale) | Yes | 0.10.5 | May. 2024 |

DDS employs a data-centric publish/subscribe architecture, facilitating reliable and efficient communication in real-time, mission-critical, and heterogeneous distributed systems [43,44]. It abstracts the complexities of communication management, reducing system coupling while enhancing dependability and flexibility through encapsulation [31,45]. DDS further promotes data interoperability by supporting multiple formats, including OMG Interface Definition Language (IDL) and Extensible Markup Language (XML). Furthermore, it is compatible with TCP and UDP transport protocols, enabling adaptable and efficient data transmission [46]. It also has extensive QoS policies that support various communication configurations, including defining data transmission intervals and enabling time-event triggers [47]. To better illustrate the evolution of DDS middleware over time, Table 4 summarizes key milestones from the initial standardization in 2004 to the latest advancements in 2024.

**Table 4.** Chronological timeline of DDS development (2004–2024).

| Year | Milestone |
|---|---|
| 2004 | First release of DDS 1.0 by OMG. |
| 2006 | DDS 1.2 Standard established; early industrial adoption begins. |
| 2007 | RTI releases Connext DDS 4.0, enhancing scalability for industrial applications. |
| 2008–2010 | DDS gains traction in defense and aerospace for low-latency communication. Early adoption in IoT and smart grids begins. |
| 2009 | First distribution of ROS released: ROS Mango Tango. |
| 2010 | ROS 1 was released. |
| 2010 | ROS (Robot Operating System) starts using DDS concepts indirectly via middleware layers. |
| 2012 | OMG releases DDS v1.4, introducing improved QoS policies and dynamic discovery. |
| 2014 | DDS Security Specification development begins. ROS 2 announces DDS as its default middleware, boosting adoption in robotics. |
| 2015 | DDSI-RTPS 2.2 published, improving real-time interoperability. |
| 2016 | DDS adopted in autonomous vehicles for real-time communication. |
| 2017 | Introduction of ROS 2, officially adopting DDS as default middleware. |
| 2018 | DDS Security v1.1 finalized, introducing authentication, encryption, and access control mechanisms. ROS 2 "Ardent Apalone" release integrates DDS, replacing ROS 1's centralized architecture. Major DDS vendors (RTI Connext, Fast DDS, OpenDDS) expand support for IoT and autonomous systems. |
| 2019 | Eclipse Cyclone DDS emerges as the default middleware for ROS 2 and becomes widely adopted in robotics. |
| 2020 | X-Types v1.3 (dynamic data models) was released, enhancing flexibility in DDS communication. DDS adoption expanded in autonomous vehicles, drones, smart grids, and cloud–edge–end fusion architectures for industrial robotics. |
| 2021 | DDS Security applied to UAV swarms and military robotics to prevent rogue node attacks. |
| 2023 | FogROS2-SGC leverages DDS for secure global connectivity in distributed robotics. |
| 2023 | Studies highlight latency-performance tradeoffs when enabling DDS security in real-time systems. |
| 2024 | Latest DDS versions (e.g., RTI Connext 7.4, Fast DDS 3.2) enable ultra-low latency and enhanced interoperability, supporting AI/ML pipelines for distributed robotic training and real-time inference. |

#### 3.4.1. Architecture of DDS Middleware

DDS architecture aims to enable efficient, real-time, and scalable data exchange between distributed systems [47,48]. Figure 5 shows the OMG standard DDS middleware protocol stack inspired by the literature [49–53]. The figure shows that the OMG DDS middleware consists of two main layers: the DDS and DDS Interoperability Wire Protocol (DDSI).

The DDS layer consists of the Data Local Reconstruction Layer (DLRL) and the Data-Centric Publish/Subscribe Layer (DCPS). DLRL is an optional extension stacked on top of DCPS, providing an object-oriented abstraction for applications that interact with DDS [32]. It also helps map raw data to structured objects, improving its usability for object-based programming paradigms [46,54]. The DCPS layer allows DDS to follow a data-centric

publish/subscribe model, where communication focuses on sharing data rather than direct connections between systems [55]. It uses publishers to send and subscribers to receive data from topics in the middleware. This approach enhances modularity and flexibility, making DDS suited for mission-critical and high-performance applications such as autonomous systems, industrial automation, and defense.

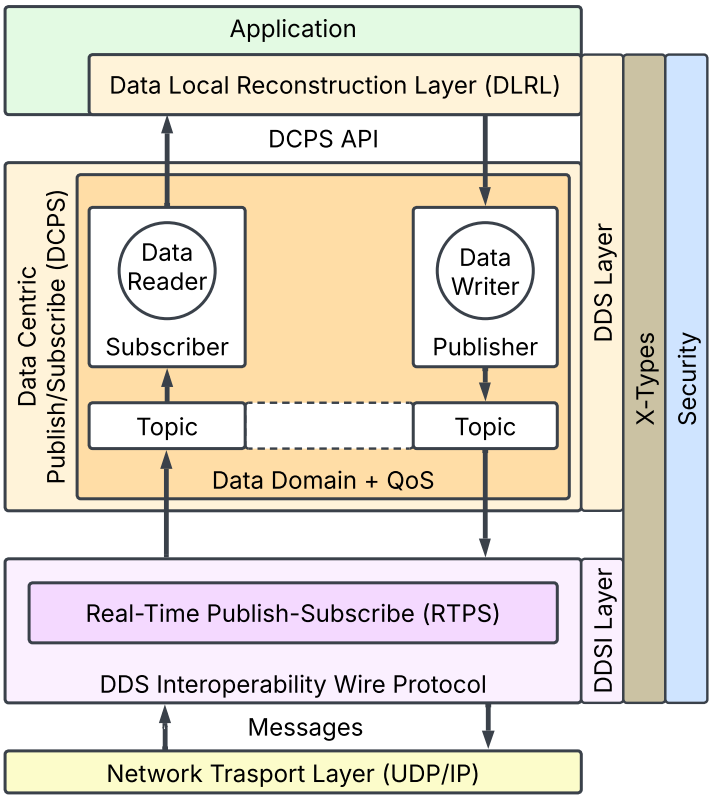



**Figure 5.** OMG standard DDS middleware stack.

The DDSI layer is a crucial component in the DDS architecture. As shown in the figure, it is an intermediary between the DDS and the transport layer, providing interoperability between different DDS implementations [51]. Furthermore, the DDSI enables different vendor implementations of DDS to communicate seamlessly, fostering a vendor-neutral ecosystem that supports multi-platform distributed systems and enhances flexibility in large-scale deployments. The Real-Time Publish-Subscribe (RTPS) protocol is in the DDSI layer. It ensures standardized and interoperable communication between DDS implementations, defining how data discovery, liveliness detection, and efficient data transfer occur in real-time systems [51,56]. It also bridges the gap between high-level DDS concepts and the underlying transport protocols [56].

Recent versions of the DDS stack have introduced several key extensions that improve interoperability, security, and extensibility. The DDS Security Specification introduces authentication, encryption, and access control mechanisms, ensuring data protection against unauthorized access and tampering [52,53,57]. The security layer is crucial for data-sensitive applications like the defense, healthcare, and financial sectors. Another key enhancement is X-Types (Extensible and Dynamic Topic Types for DDS) [52,53], which provides dynamic data modeling capabilities, enabling DDS applications to evolve without breaking compatibility with existing systems. This feature is essential for long-term system maintenance.

Having outlined the DDS protocol stack, we now discuss its practical deployment and architectural structure. DDS enables a distributed and data-centric communication

model where multiple participants interact through Topics within a shared Global Data Space. This approach ensures efficient and decoupled data exchange, making it ideal for real-time and mission-critical applications. The following represent the key architectural components of DDS and as shown in Figure 6:

- Domain: This is a logical concept representing a group of applications that can communicate with each other. A DDS system can have multiple domains, each representing different sets of applications that interact with each other and consist of one or more Domain Participants [32].
- Global Data Space (GDS): The GDS in DDS is a core concept that serves as a virtual, fully distributed, and shared space where all data exchanges occur between distributed entities [55]. It allows publishers to write data and subscribers to read it without direct connections, thus decoupling them in time and space [54]. The GDS enables the system to become scalable and eliminates the risk of a single point of failure [58].
- Domain Participants: They represent local membership of the applications within a specific domain and can participate in multiple domains simultaneously [32]. Each Domain Participant can contain a Publisher and a Subscriber [43].
- Publisher: The Publisher is the middleware entity responsible for publishing data written by data writers on their topics and pushed to the GDS [59]. A publisher may contain multiple data writers. However, a data writer can write on one particular topic.
- Topic: The Topic facilitates the communication between data readers and writers. It specifies a unique name, a data type, and a set of QoS policies for published or subscribed data [32]. A single topic can be linked to multiple publishers and subscribers and has several instances, each identified by a unique key [46].
- Subscriber: The Subscriber is the middleware entity that receives and consumes published data of interest exchanged in the GDS via its corresponding data reader [43,59]. The relationship between the subscriber and the data reader can be one-to-many. However, the data–reader–topic relationship is one-to-one.

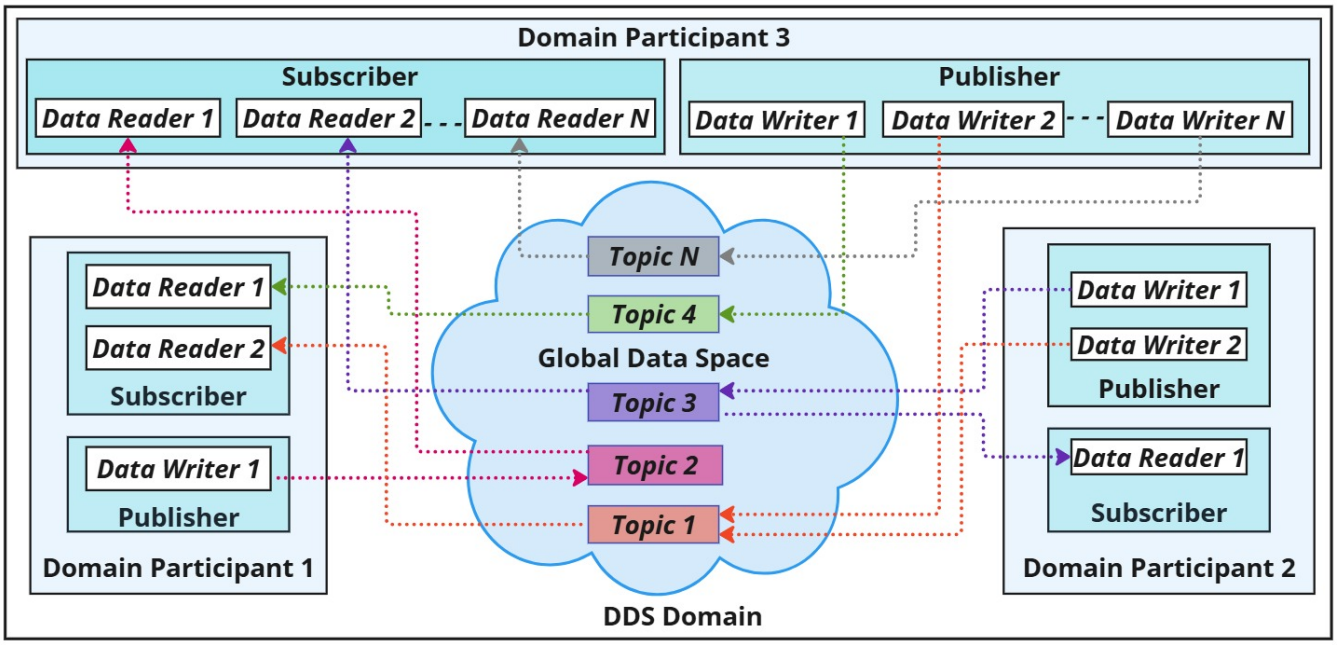



**Figure 6.** Architecture of DDS middleware.

### 3.4.2. Quality of Service (QoS) in DDS Middleware

Quality of Service (QoS) policies in DDS middleware are a set of configurable parameters that control various aspects of data communication between publishers and

subscribers [47]. They help ensure that DDS systems meet the specific requirements of distributed real-time applications. DDS provides 22 distinct QoS policies and allows each entity to define its own set of QoS parameters [46]. Also, communication between applications is possible only when the topic names and the specified QoS parameters align [32]. QoS policies define various aspects of data handling, such as delivery, availability, timeliness, resource management, configuration, and the lifecycle of entities. These include policies for setting resource limits (e.g., sample count, instances, and instances per sample) on data queues, durability, reliability, latency budget, ownership, deadlines, lifespan, and more [60]. Also, QoS settings can define whether the DDS system will guarantee delivery of the most recent published data, the complete history of published data, or something in between [61]. More information on DDS specifications for QoS is available on the OMG website [24].

#### 3.4.3. Security in DDS Middleware

The DDS middleware is one of the most widely used communication architectures for the real-time interactions it enables, making it an essential part of robotic systems and distributed applications [43,49]. However, its applications in various sensitive and mission-critical infrastructures come with high-security requirements, partaining confidentiality, integrity, and availability of data [62]. The DDS Security Specification defined by the OMG focuses on securing DDS communications in response to key vulnerabilities and allowing scalable protection mechanisms [63,64].

DDS security has many applications in autonomous systems, collaborative robots, and unmanned vehicles in robotics and IoT applications to secure communication. Although DDS security mechanisms improve the resilience of systems in general, they come with performance overheads [43]. Research has shown that enabling security features, such as encryption and authentication, adds significantly more latency and less throughput to high-frequency communication scenarios [65]. Therefore, balancing security and real-time requirements is still a key concern. DDS security evolution now emphasizes adaptive and lightweight security protocols, reducing the impact on performance while ensuring robust protection.

Figure 7 shows the DDS security architecture according to OMG specification [57]. Some of the key features of DDS security include the following:

- Authentication: DDS ensures that only authorized entities participate in the communication by employing authentication mechanisms such as X.509 certificates [66]. These certificates validate the identities of the publishers and subscribers, preventing unauthorized access to the data stream [57].
- Access Control: The middleware applies fine-grained policies to restrict data access at all levels, like topics, partitions, and QoS policies [43]. Predefined governance files signed by a Certificate Authority (CA) enforce these policies [66].
- Encryption: DDS implements cryptographic technologies based on the Advanced Encryption Standard (AES) in combination with relevant techniques to keep data private during transmission [67]. The middleware encrypts the payload, protecting the data from disclosure and man-in-the-middle attacks [57].
- Data Integrity: The integrity of transmitted data is verified by digital signatures and Message Authentication Codes (MACs) [57]. This strategy prevents messages from being changed or modified in transit, increasing the trustworthiness of communications.

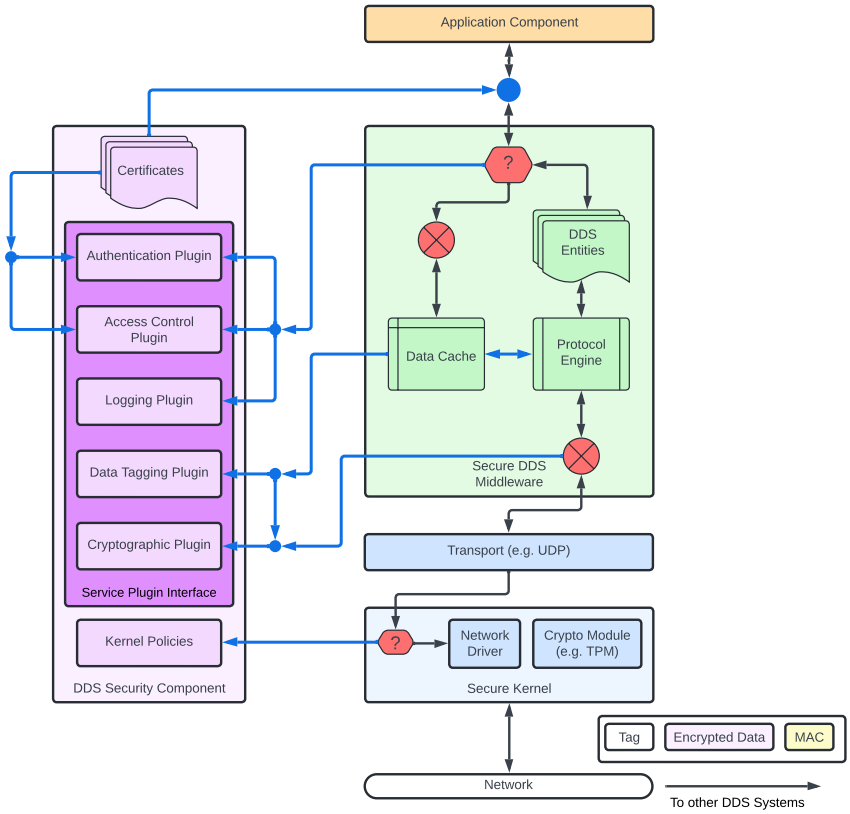



**Figure 7.** OMG DDS security architecture.

## 4. DDS Middleware Applications in Robotics

DDS middleware is crucial in modern robotics by enabling efficient, scalable, and real-time communication across diverse robotic systems. DDS has many applications in robotic systems: It is used as the primary middleware for reliable and real-time data exchange or as an embedded component within middleware solutions such as the ROS [68,69]. It ensures seamless interoperability across diverse robotic applications, including swarm robotics, industrial automation, autonomous vehicles, and healthcare robotics, regardless of the vendor or system architecture [61,70]. Its versatility allows it to be integrated into various robotic architectures, communication models, and application domains, addressing key challenges such as interoperability, decentralized control, and dynamic system integration. The publish–subscribe paradigm inherent in DDS enhances adaptability, making it suitable for diverse robotic applications, from industrial automation to autonomous systems.

To provide a structured analysis, we categorized DDS applications in robotics based on key aspects identified from the 41 selected papers. As shown in Figure 8, these applications can be classified into three main categories: communication technique, which categorizes communication technologies and models of DDS-based robotic systems; application types, which refers to why and where scientists use DDS in robotics; and architecture, which discusses the different architectures researchers used in the DDS-based robotic system. Each dimension highlights how DDS facilitates efficient data exchange, real-time processing, and seamless integration in robotics. The following subsections explore these categories in detail, demonstrating the impact of DDS in multi-robot systems, service robots, industrial automation, and other critical domains.

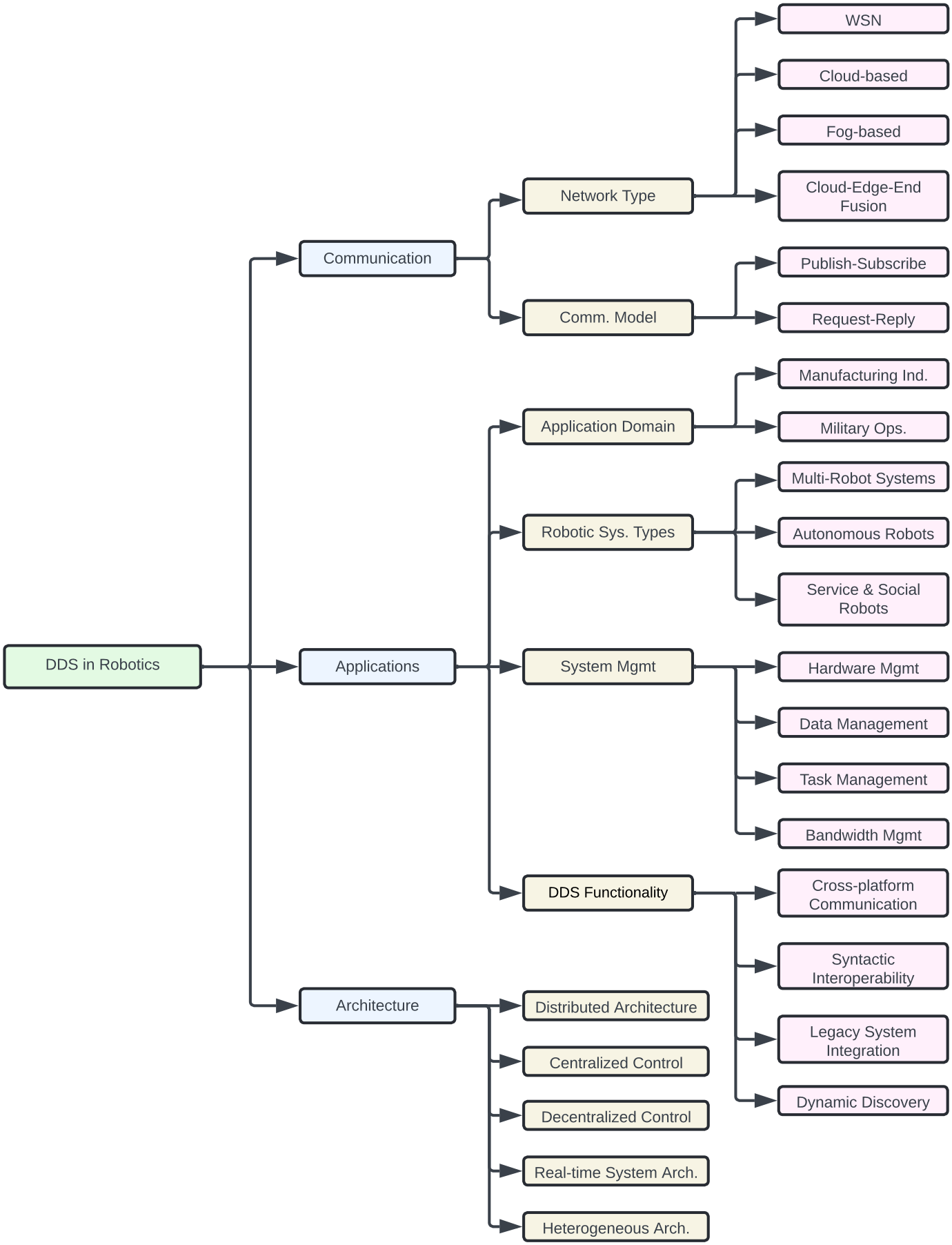



**Figure 8.** Taxonomy of DDS applications in robotics.

### 4.1. DDS-Based Communication Techniques in Robotics

One of the most critical aspects of DDS in robotics is its impact on communication technology. Effective communication is essential for robotic systems to coordinate actions, exchange sensor data, and operate in distributed environments. DDS enables low-latency, high-reliability communication by supporting diverse network architectures and communication models, ensuring that robotic applications can function seamlessly across various deployment scenarios. The selected papers show that the applications of DDS middleware in robotic communication technology can further be classified based on the network type or the communication model. On the one hand, the network types (cloud, fog, hybrid, and WSN) significantly impact DDS-based robotic communication by offering diverse processing power, latency, and scalability options. On the other hand, communication models (publish–subscribe and request–reply) offer different message structures and transmission techniques suited to different robotic communication needs, ranging from event-driven to direct interactions.

#### 4.1.1. Classification by Network Type

In robotic systems, the network type used for communication significantly impacts the performance and efficiency of the system, especially in distributed and dynamic environments. DDS has a variety of network types for system integration.

Wireless Sensor Networks (WSNs)

WSNs are networks of resource (such as processing power, energy, and memory) constrained computing devices for fine-grain sensing [71]. WSNs represent another network type commonly utilized in robotics, particularly in swarm and collaborative robotic applications that operate in remote or infrastructure-limited environments. WSNs enable low-power localized communication between robotic devices, optimizing efficiency in distributed robotic systems. Seungwoo et al. [72] proposed a method to program multi-mobile robots using behavior trees (BTs). BTs are a modular and flexible method for controlling robot behavior, initially developed in the computer gaming industry [73]. The authors used DDS with ROS 2 to enable communication between multi-mobile robots and servers for task allocation and monitoring. This implementation allows robots to share sensor and actuator data efficiently by hiding the hierarchical structure of IP addresses, making it easier for robots and servers to connect without requiring manual network configurations.

Cloud

However, WSN-based robotic systems often suffer from limited bandwidth and processing power, restricting their ability to handle complex, data-intensive tasks. Thus, researchers move processing to the cloud to solve these issues. Cloud-based networks offer high processing power and storage capacity, making them ideal for handling large datasets and analytics. Hartanto et al. [74] proposed a cloud-based DDS communication framework that enables robots to synchronize tasks and share critical data, such as sensor readings, facilitating reliable communication between multi-robot systems during cooperative tasks such as mapping. Using DDS as the transport layer for communication, the authors demonstrate that the approach significantly reduces message loss even with temporary communication disruption. Another benefit of cloud-based solutions is that they allow researchers to move complex computations to servers, reducing the hardware and energy overhead.

Fog Computing

However, robots require constant or periodic communication with the cloud, which is unsuitable for remote applications due to high latency. Furthermore, some computing and communication needs, such as location awareness, mobility support, and low latency, are susceptible to inefficiency due to the growing demand for cloud computing services from resource-limited devices like IoT devices [75]. Some scientists use fog computing to compensate for these constraints. Fog-based networks find applications in areas with high bandwidth, security, reliability, and low latency requirements [76]. Fog networks enable data to be processed near the robot, thus improving latency and responsiveness, which are crucial for robotics applications such as autonomous vehicles.

The study by [77] demonstrated DDS as a middleware solution to enable syntactic interoperability between Robot Operating System 2 (ROS 2) and Adaptive AUTOSAR (AUTomotive Open System ARchitecture). The authors proposed using DDS in fog computing, where computing tasks are offloaded from the cloud to edge nodes due to DDS features such as reliability, scalability, and fault tolerance.

Recently, researchers have developed middleware for fog-computing-based robotic systems [78,79]. Jeffrey et al. [78] introduced FogROS2, an extension of ROS 2 that enables secure, scalable, and low-latency cloud and fog computing integration for robotic systems, allowing robots to offload computation to edge or cloud resources while maintaining real-time performance. Kaiyuan et al. [79] developed an extension of FogROS2, called Fog-ROS 2-Secure Global Connectivity (FogROS2-SGC), which enables global secure connectivity for robots in different physical locations and networks. The authors claimed that FogROS2-SGC is DDS-agnostic, which means it can provide services on any ROS 2 distribution with any network transport middleware without modification. The authors also tested the system using four robots that were 3600 km apart. They recorded a 19-fold increase in speed compared to traditional methods.

Cloud–Edge–End Fusion

A hybrid cloud–edge–end fusion model optimizes robotic systems, where the cloud layer performs heavy processing tasks, the edge computing layer acts as an intermediate storage and processing layer [80], and the end devices (robots) perform real-time tasks. This approach balances low-latency processing near the robot with the scalability of cloud resources, making it ideal for large-scale, latency-sensitive operations such as smart factories [81,82]. An example is seen in [83], which uses DDS as its communication middleware for cloud–edge–end fusion robots. The proposed method involves a hybrid switching system model consisting of priority-based and time-based subsystems to improve the scheduling of ROS 2-RTPS messages. The authors aimed to reduce packet loss, improve real-time performance, and ensure high-priority data transmission in complex robotic applications. Integrating DDS within cloud–edge–end fusion models enables efficient data distribution while optimizing resource utilization and real-time responsiveness in industrial and autonomous robotics applications.

#### 4.1.2. Classification by Communication Model

The communication model determines how data flows within the robotic system, impacting scalability, responsiveness, and control. While publish–subscribe is the primary model in DDS, the middleware also supports request–reply communication patterns [84] to accommodate diverse requirements. Publish–subscribe enables asynchronous, many-to-many communication, while request–reply facilitates on-demand, direct interactions.

Publish–Subscribe Model

The publish–subscribe model is a communication model supported by DDS middleware (See Section 3.4), where robots publish data to multiple subscribers, enabling efficient and asynchronous communication without overloading the network. This model organizes data by topics, allowing nodes to focus on specific data topics without concern for the sender or receiver and thereby ensuring loose coupling in data transmission [55]. Many studies use the publish–subscribe model as the primary communication model for real-time data exchange between robotic nodes [55,85–90]:

In [85], DDS was used to develop fpgaDDS, a novel intra-FPGA data distribution service designed for ReconROS applications. In the proposed system, the hardware nodes are responsible for publishing the data. These nodes (publishers and subscribers) connect to Hardware-Mapped Topics (HMTs) via Advanced eXtensible Interface-streaming (AXIS) networks. The publishers send data to the HMTs, which then distribute the data to the subscribers. The fpgaDDS benefits real-time, low-latency, and high-throughput applications in hardware-constrained environments, such as autonomous vehicles and industrial automation. However, its hardware dependency, development complexity, and limited scalability beyond FPGA architectures restrict its broader applicability in general-purpose robotic and distributed computing systems.

Microcontrollers are more popular than FPGAs in embedded systems, real-time systems, and robotics due to their lower cost, ease of programming, and sufficient processing power for typical tasks. As a result, DDS implementations on microcontrollers have become more accessible and practical across a wide range of applications, from simple embedded systems to complex robotic projects. Kampmann et al. [87] introduced embeddedRTPS, a portable open-source implementation of the RTPS protocol for microcontrollers. The microcontrollers are the participants, comprising writers and readers. They discover each other through a simple discovery process. After the discovery process and setup, writers publish data on a specific topic, datatype, and QoS, and the readers receive them accordingly.

Our research shows that publish–subscribe middleware is more commonly used for connecting higher-level system components in robotics (nodes, servers, clients, sensors, and actuators) than at the FPGA and microcontroller levels due to the following factors: Firstly, these higher-level components often require more complex, flexible, and scalable communication patterns that publish–subscribe models excel at providing; secondly, middleware facilitates efficient data distribution, component decoupling, and easy integration of various subsystems, which is crucial to managing the complexity of robotic systems.

DDS enhances communication performance in various robotic applications. In [86], a publish–subscribe model was used to convert the Flexible Behavior Engine (FlexBE) and the Flexible Navigation system to ROS 2, where DataWriters sent data to the GDS, and DataReaders retrieved it.

Request–Reply Model

Another vital communication model supported by DDS is the request–reply model. A request–reply model is a type of communication in which one application sends a request and another responds. Figure 9 demonstrates how a Requester and a Replier exchange messages through DDS topics [84]. The Requester initiates communication by sending a Request message to the Request Topic, which forwards it to the Replier. Upon receiving the request, the Replier processes it, generates a Reply, and sends it to the Reply Topic. Finally, the Requester retrieves the reply from the Reply Topic, completing the interaction. This model enables asynchronous, decoupled, and reliable communication, making it ideal for distributed robotics and real-time systems where on-demand responses are required.

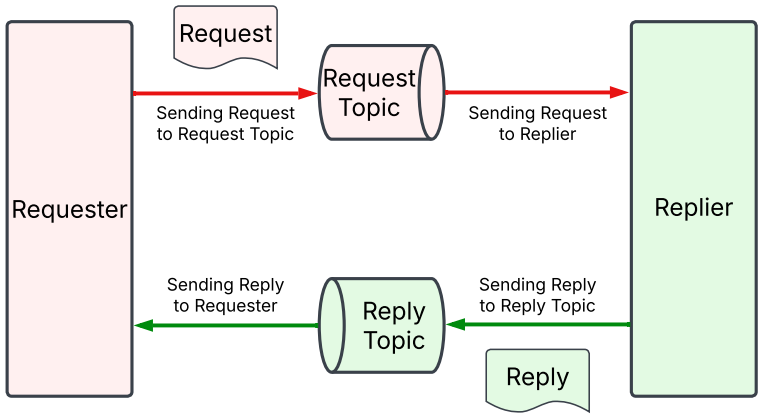



**Figure 9.** A typical request–reply DDS model.

An example of a request–reply model scenario is an application where robots or sensors need on-demand access to data or services, such as a robotic arm requesting updated instructions from a central controller in response to detected changes in task parameters [91]. In [55], Xiaowen et al. present a service integration framework called Dynamic-DDS-RPC for industrial robots. The framework integrates a request–reply mechanism within DDS, using JSON for service description and dynamic library loading to support dynamic service discovery and invocation. Initially, the robots perform service registration by reading and locally storing the configuration files. After registration, they follow the four-step request–reply mechanism: First, a user on the distributed nodes sends a service request that includes details such as the service name and parameters. Next, a sender transmits the request through a specific service topic within the data domain. The receiver then executes the requested service based on the provided parameters. Finally, the middleware returns the execution result to the requester through a corresponding response topic. This mechanism ensures low latency and efficient communication.

### *4.2. Applications of DDS in Robotics*

The DDS is a crucial component in robotics because it offers a reliable and extensible communication infrastructure that can accommodate many robotic systems with high interoperability and real-time data exchange. Hence, DDS has become an essential tool in many robotics applications. After a general overview of these applications, we classify DDS applications in robotics into four areas: Application Domains, which highlight the various sectors that use DDS-based robotic systems; Robotic Systems Types, which cover specific classes of robotic systems that rely on DDS for communication, such as autonomous and multi-robot systems; System Management, which focuses on how DDS is used to manage resources, monitor performance, and ensure reliable operations; and DDS Functionality, which addresses the core features of DDS that enable robust communication in robotic environments.

#### 4.2.1. Classification by Application Domain

In various applications, the DDS framework has become indispensable for enabling robust and seamless communication between robotic systems. The adaptability and real-time capabilities of DDS have supported its adoption across diverse fields, each with unique requirements and challenges. Some of these application domains include the following described below.

##### Manufactory Industry

DDS-based factory automation has significantly altered industrial machine-to-machine (M2M) communication, allowing high performance in real-time data transfer essential for

dependability and efficiency. A recent study introduced a DDS-based middleware for factory automation. It focuses on ensuring smooth connectivity among devices such as robotics systems, industrial PCs (IPCs), and Programmable Logic Controllers (PLCs) [70]. This middleware framework takes advantage of the extensive DDS QoS capabilities to simplify implementation for users with limited DDS experience. It addresses the unique demands of distributed control systems by providing a structured approach to managing communication types and QoS parameters across diverse devices.

In addition to factory automation, DDS is equally effective in remote communication. Sundarapandian et al. [92] developed TeleBot, a communication architecture for effective human–robot interaction. TeleBot can react to voice commands and stream live videos using its voice recognition system and high-definition cameras. It communicates effectively with the remote operator using DDS middleware as the communication channel connecting the operator interface and the TeleBot interface. With the help of the modular communication model of DDS, any input device can publish data as articles, enabling simultaneous subscriptions from several devices. The design significantly improves the operational reliability and performance of the system.

In the context of industrial communication applications, the research shows that Dynamic-DDS-RPC outperformed traditional web service and DDS-RPC middleware due to its support for dynamic library loading, which enables service discovery and invocation in multi-robot real-time control systems [55]. This approach combines agile service configuration with dependable real-time communication.

Military Operations

Most military operations are mission-critical. Recent research [90] highlights DDS as a critical middleware solution in improving the interoperability and cybersecurity of advanced robotic and autonomous systems. It addresses interoperability challenges in multi-robot systems and legacy Command and Control (C2) systems. The authors have demonstrated how DDS middleware improves data exchange and system integration, fostering adaptability, resilience, and scalability in military operations.

Similarly, recent wars have unlocked many military applications of UAVs [93]. The authors in [94] evaluated the enhancements in ROS 2, which integrates DDS for real-time, mission-critical communication. The study highlights the effectiveness of DDS in safeguarding UAV communications against cyber threats, such as rogue node attacks in UAV swarms. While addressing the tradeoff between increased security and latency, the research showcases how DDS can support real-time and secure communication in distributed systems essential for military applications.

#### 4.2.2. Classification by System Management

System management is another area where DDS is crucial, as it is necessary to manage robotic systems. DDS guarantees dependable operations, real-time monitoring, and effective resource allocation. It maximizes resource utilization while preserving fault tolerance and high performance through component coordination. DDS is excellent at facilitating the dynamic distribution of resources among robotic subsystems, offering flexible and scalable system administration.

Hardware Management

A prime example of middleware for hardware management is fpgaDDS [85], achieving 13.34 times faster execution time than conventional software-based solutions. It also ensures a predictable real-time behavior by reducing memory access overhead by mapping data transport directly onto the FPGA fabric. Furthermore, there is no need to modify current ROS nodes, because fpgaDDS easily integrates with the ROS 2 publish–subscribe model.

Enhancing flexibility by allowing hardware nodes to communicate with software and hardware nodes, the framework supports QoS, which guarantees dependable communication between hardware and software components. This method demonstrated that hardware management can maximize the performance and dependability of robotic systems.

Similarly, the system in [95] is another example of hardware management in robotics, using ROS 2 and micro-ROS integration to efficiently coordinate inexpensive, resource-constrained components, like the Raspberry Pi 4 and YD Lidar X4. Real-time communication via DDS-based middleware allows for effective hardware resource management, enabling concurrent obstacle avoidance, navigation, and Simultaneous Localization and Mapping (SLAM) tasks [95]. This method demonstrates a scalable solution for inexpensive autonomous robotic systems by optimizing the interaction and control of multiple hardware components.

Data Management

DDS plays a crucial role in the data management of robotic systems by ensuring efficient, real-time, and scalable communication between distributed robotic components. As a data-centric middleware, DDS provides reliable, structured, and quality-controlled data flow, optimizing information exchange across sensors, actuators, and control systems.

Jalil and Kobayashi [96] introduced the Cache-Control Algorithm (CCA) as part of their low-cost multi-robot systems network architecture, Aggregated Robot Processing-Robot Operating System (ARP-ROS). It vastly reduces latency and data processing failures in multi-robot systems (MRSs), improving data management. DDS-based middleware ensures real-time communication and reduced message loss, which enables effective data exchange between robots and a dedicated computing environment (Computing Environment Dedicated to Data Processing (CEDDP)). The system is made for low-cost multi-robot systems and provides scalable solutions that let robots efficiently share processing power. The proposed system improves communication efficiency and data management in robotics applications with limited resources.

Likewise, in [97], the authors used DDS middleware for real-time communication. The authors simulated a system for interactive path editing and motion planning of collaborative robots. They integrated Unity 3D and ROS 2 to enable real-time control and synchronization of robot trajectories in hazardous environments. The system leverages collaborative robots to engage in interactive path editing and simulation. By enabling dependable and effective data exchange between the robot control system and the simulation environment, DDS is essential to data management. It guarantees that trajectory data are transmitted in real-time from the host device to the control node, enabling precise robot action synchronization. For safe operation in dangerous environments, this strong data management guarantees that trajectory updates are applied in the simulation and control systems as soon as possible, preserving consistency and avoiding data loss.

Task Management

DDS plays a critical role in task management for robotic systems through decentralized real-time task coordination between distributed robotic components. As a middleware framework, DDS enables dynamic task allocation and real-time synchronization.

Task management is crucial for efficient coordination and the execution of tasks in multi-robot systems. Essers and Vaneker [61] offered a task planning framework that effectively manages several robots using DDS. The system allows robots to communicate asynchronously thanks to BTs, which improves their capacity to work together in dynamic environments. DDS uses decentralized communication to manage tasks, and robots can join or exit the system at any time, providing flexibility in task completion without the need

for a centralized control unit. Also, the data-centric nature of DDS supports a decentralized and fault-tolerant approach, which maximizes task management by guaranteeing seamless coordination and communication between robots in complex environments.

Cloud-based DDS (See Cloud in Section 4.1.1) is a viable task management solution for robotic systems. It enhances task coordination and fault tolerance through decentralized data synchronization with the cloud and real-time information sharing between robots [74]. This configuration guarantees that robots can continue working together effectively when communication links with the cloud momentarily break. DDS middleware improves task management by facilitating dynamic task execution, encouraging robustness in multi-robot operations and facilitating real-time updates, which results in more effective task execution in distributed environments.

Bandwidth Management

Efficient bandwidth management is crucial in robotic systems, ensuring real-time communication, minimal latency, and optimal network performance. DDS is vital in optimizing bandwidth usage by intelligently controlling data flow, prioritization, and network efficiency through its QoS policies and data-centric communication model.

To further explore the management of bandwidth in distributed robotic systems. Lourenço et al. [98] bridged a gap in the literature by practically integrating ROS with Kafka, which advances the field. This bridge code permits smooth data exchange by guaranteeing dependable communication between streaming platforms and robotic systems in smart warehouses. It tackles bandwidth management issues using QoS profiles from DDS, improving fault tolerance, scalability, and performance. The solution efficiently handles massive data streams while maintaining high communication reliability. The work offers a strong foundation for effective communication in distributed robotic systems, especially smart warehouses.

#### 4.2.3. Classification by Robotic System Types

Robotic systems can be classified according to their structural and operational characteristics, which define their capabilities and applications. This survey found that the selected papers discuss three robotic system types: autonomous robots, which operate independently without human intervention; multi-robot systems, which coordinate multiple robots to complete tasks collaboratively; and service and social robots, which assist humans by performing tasks in environments such as homes, hospitals, and factories. Each of these systems has unique characteristics and applications that contribute to the advancement of the field. The following subsections provide an overview of how DDS is used in different robotic system types.

Autonomous Robots

Autonomous robots are sophisticated machines capable of performing tasks without human intervention, utilizing advanced sensors, artificial intelligence, and complex algorithms to navigate and interact with their environment [99]. These robots are designed for self-maintenance, environmental sensing, and task optimization, making them adaptable to various industrial applications, such as manufacturing, logistics, and healthcare. A notable example of DDS application in autonomous robotic systems is UAVs for military applications, enabling real-time communication and security [94].

The research in [58] examines specialized callback-group-level executor usage in ROS 2 to advance real-time capabilities in autonomous robotic systems. Through a ping pong test bench, the study evaluated the efficacy of substituting a real-time-focused executor for the default, demonstrating improved deterministic execution. With its data-centric publish/subscribe model, the DDS middleware facilitates dependable low-latency data transfer

crucial for autonomous applications such as self-driving cars. As a step toward reliable real-time performance for autonomous systems, the study offers insightful information on improving real-time communication in ROS 2.

In the same way, Teper et al. [89] addressed the critical issue of end-to-end latency in ROS 2 systems within autonomous driving applications, proposing an optimization approach for employing a custom scheduler for task execution using constrained programming. The study analyzed multi-executor setups and cause-effect chains to minimize delays in real-time communication, a key factor for the efficiency and safety of autonomous robots. The authors used DDS-based middleware for real-time data transfer. The approach leverages a publish–subscribe model to streamline inter-node communications and ensure precise timing across system components. They tested the solution on an autonomous driving software stack, where the optimization method demonstrates up to a 50.2% reduction in latency, highlighting its effectiveness in enhancing performance and predictability in complex robotic systems like autonomous vehicles.

Service and Social Robots

A service robot improves efficiency and quality in various industries by assisting humans and reducing costs [100]. These robots enhance service delivery and operational effectiveness in healthcare, hospitality, transportation, education, and other fields. Social robots interact with humans in an engaging, relational manner by simulating behaviors like communication, empathy, and companionship. They are used in education, therapy, and companionship to build trust and rapport with users [101]. We grouped the two into one section because they aim to assist humans, utilize similar technologies, and often overlap in applications like healthcare, education, and customer service, where efficiency and human interaction are essential.

Jo et al. [102] presented a scalable software architecture designed for service robots using ROS 2. It aims at improving task management and device integration. The architecture leverages Python 3.8 with multi-processing to enhance both performance and ease of integration, allowing service robots like the SeoulTech Service Robot (SSR) to efficiently perform tasks such as navigation, SLAM, and AI-based applications. DDS-based middleware enables the system to support real-time operation, security, and reliable performance. It effectively manages tasks through multi-threading and multi-processing and simplifies the integration of diverse devices, making it highly scalable and adaptable for various service robotic applications.

Regarding social robots, Cruz et al. [103] developed “Nerve” as a lightweight C++ middleware to ensure scalability and QoS for real-time applications by facilitating code reuse and networked task deployment. They showed how well it can improve the cognitive abilities necessary for social robots, such as learning by imitation and the visual attention mechanism. Nerve integrates DDS with well-known design patterns, allowing it to achieve scalability and implement a data-centric publish/subscribe model that facilitates high-performance real-time communication. In social robotics, where real-time adaptability to human interactions is crucial, the QoS policies of the middleware are particularly helpful in managing dynamic interactions and guaranteeing system reliability. Nerve also encourages code reuse, making it easier to deploy networked tasks across different applications.

Both studies highlight the significant role of DDS-based middleware in enhancing the performance and scalability of robotic systems. Whether in service robots, as seen in the SSR, or social robots, through the Nerve middleware, DDS enables real-time communication, reliable task management, and improved cognitive capabilities. These advancements provide valuable insights into developing more efficient, adaptable, and reliable robotic applications across various domains.

Multi-Robot Systems

A multi-robot system (MRS) consists of multiple robots collaborating to complete tasks, providing advantages such as improved time efficiency and reduced risk of single-point failures compared to single-robot systems [104]. These systems have applications in surveillance, search and rescue, and transportation, where effective coordination and communication among robots enhance overall performance.

MRSs rely heavily on effective communication and coordination for seamless collaboration and efficient task execution. In this regard, Deshpande and Kamalanathan [105] investigated how DDS-based middleware improves the performance of MRSs, emphasizing path planning and interoperability. The authors integrated the Robotics Middleware Framework (RMF) with Free Fleet. They used the resulting system to test three path planners—(1) Dynamic Window Approach (DWA) [106], (2) Timed-Elastic-Band (TEB) [107], and (3) the Regulated Pure Pursuit (RPP) Controller [108]—in real-world settings to identify the best planner for complex scenarios. The DDS middleware makes it easier for robot fleets to communicate with one another, guaranteeing seamless data exchange, task synchronization, and scalable and dependable multi-robot operations.

Dey et al. [109] introduce an ROS 2-based synchronization middleware designed to optimize communication between heterogeneous MRSs within mission-critical IoT applications. Using DDS and a custom QoS policy, the middleware minimizes packet loss and latency, enhancing the reliability and speed of data exchange between various robotic agents. This setup is particularly suitable for scenarios such as search and rescue, wildfire monitoring, and disaster assessment, where multiple robot types must collaborate effectively. The DDS facilitates synchronized, low-latency communication, ensuring that essential data are transmitted reliably in complex and critical environments and thereby enhancing coordination and responsiveness in multi-robot systems.

#### 4.2.4. Classification by DDS Functionalities

The selected papers show that DDS provides powerful functionalities essential in efficient robotic systems, ensuring flexibility, scalability, and efficiency in distributed environments. These key functionalities of DDS include syntactic interoperability, which standardizes data formats for seamless communication; legacy integration, which allows the easy addition of newer components to the robotic system; dynamic discovery, which enables automatic detection and connection of new nodes without manual configuration; and cross-platform communication, which ensures compatibility between different hardware and software environments. The following sections discuss them in detail.

Cross-Platform Communication

DDS facilitates interoperability across dissimilar communication platforms. For example, a middleware like FogROS2 and FogROS2-SGC enables secure and reliable communication between robots across various software systems, physical locations, and network protocols [78,79].

Research also shows that a robotic system can have multiple sets of middleware that communicate with each other. The study in [110] examined cross-platform communication between three prominent DDS middleware systems: Fast DDS [23], Cyclone DDS [42], and RTI Connext [37]. They experimented with and without the security feature. Table 5 shows their results. The table shows that all three middlewares are compatible without security. However, only Fast DDS is compatible with the other middlewares when the security feature is enabled. The authors also found that the security feature increases latency, especially with larger message sizes.

**Table 5.** Compatibility between three DDS middlwares.

| | | Publisher | | |
|---|---|---|---|---|
| | | **Fast DDS** | **Cyclone DDS** | **RTI Connext** |
| Subscriber | Fast DDS | ✔ ⊙ | ✔ ⊙ | ✔ ⊙ |
| | Cyclone DDS | ✔ ⊗ | ✔ ⊙ | ✔ ⊗ |
| | RTI Connext | ✔ ⊗ | ✔ ⊗ | ✔ ⊙ |

✔ Compatible Without Security
⊙ Compatible With Security
⊗ Incompatible With Security

Additionally, the researchers in [105] used RMF of ROS 2 as middleware to integrate robots with the open-source robot fleet management system, called Free Fleet [111]. DDS plays a crucial role in the paper as part of the communication stack in ROS 2, which serves as the foundation for RMF. It provides essential security features, including authentication, encryption, and access control, ensuring secure and reliable communication between robots and systems. Also, DDS enhances interoperability, enabling RMF to effectively manage multi-robot systems and seamlessly integrate with other fleet management solutions like Free Fleet.

Syntactic Interoperability

Syntactic interoperability refers to the ability of different systems to exchange data using compatible formats and protocols [112]. It is crucial in middleware-based systems that enable seamless communication between heterogeneous components. The middleware is an intermediary layer that facilitates data translation and protocol conversion, ensuring that systems can interact effectively despite differences in native data formats and communication standards.

DDS supports syntactic interoperability. Researchers explored the integration of ROS 2 with Adaptive AUTOSAR software using DDS as a middleware [77] (see Fog Computing in Section 4.1.1). They demonstrated the capability of DDS to ensure reliable delivery of messages across different platforms and languages, which is essential for integrating diverse systems in automotive and other smart applications.

Legacy System Integration

Legacy integration in robotic systems is essential for ensuring interoperability with existing infrastructure, reducing costs, and maintaining operational continuity. Many industries, such as manufacturing, healthcare, and logistics, rely on older robotic systems that must coordinate with newer technologies without complete replacement. Upgrading legacy systems can be expensive, so integration allows for a gradual transition while preserving proprietary software, regulatory compliance, and existing workflows. Also, legacy robots often need to interact with modern IoT devices, AI-driven controllers, and cloud-based systems, requiring middleware solutions like DDS to bridge communication gaps. Legacy integration ensures efficient, cost-effective, and future-proof robotic operations by enabling scalable and secure interoperability.

Integrating new systems with legacy components remains a significant challenge in robotics. Fedi and Nasca [90] proposed the Simulated Interactive Robotics Initiative (SIRI). It is a framework that leverages BEE-DDS for real-time data communication and the National Information Exchange Model (NIEM) to improve interoperability between multi-robot fleets and legacy C2 systems. Their approach addresses data exchange and data model interoperability challenges, introducing the concept of "tri-lateral interoperability" among robotic automation systems, C2 infrastructures, and simulation platforms. SIRI

combines DDS and NIEM to mitigate common issues such as vendor dependency, high maintenance costs, and integration complexity. The framework demonstrates how DDS can enable seamless collaboration between autonomous and semi-autonomous robots while maintaining compatibility with legacy systems, highlighting DDS's critical role in enhancing communication and coordination in heterogeneous robotic environments.

Dynamic Discovery

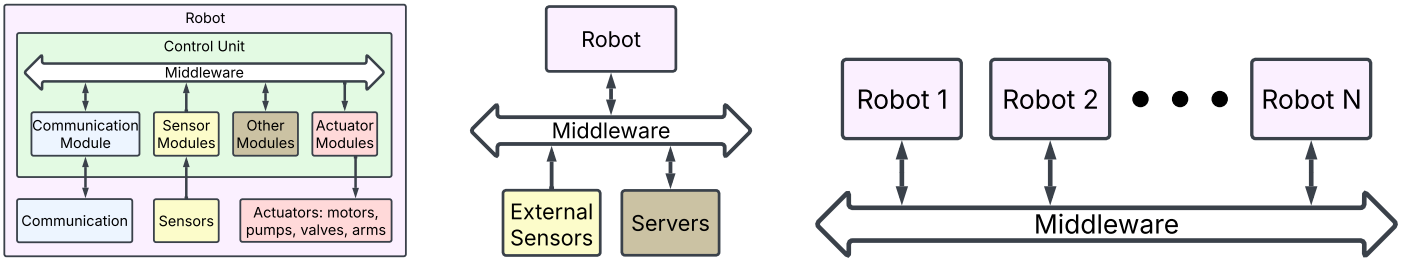
Dynamic discovery is a feature of DDS that allows robotic systems to automatically detect, recognize, and communicate with new nodes or devices without requiring manual configuration. Thus, robots, sensors, and control systems can dynamically join or leave a network without disrupting communication. It enhances scalability, flexibility, and adaptability, making it useful for multi-robot coordination, autonomous navigation, and distributed robotics. It also enables robots to self-organize in dynamic environments, such as search and rescue missions, industrial automation, and swarm robotics, where real-time adaptability is crucial.

Additionally, it reduces setup complexity by eliminating the need for predefined network configurations, making robotic deployments more efficient. Dynamic discovery is crucial in autonomous decision making, decentralized control, and real-time data exchange in robotic ecosystems, ensuring seamless communication and automatic peer detection. A prime example of DDS dynamic discovery from the reviewed literature is presented in [113]. The authors proposed a BT-based task planning system for interacting via DDS. This framework enables multiple robots to discover each other dynamically and share data asynchronously. The system exhibits enhanced reactivity in dynamic situations to ensure reliable operations. It also allows robots to enter recovery modes when they encounter errors.

### *4.3. DDS Based on System Architecture*

This category classifies DDS-based robotic systems according to the overall system architecture containing the robots and defining their structural organization, control mechanisms, and real-time capabilities, which are crucial for scalability, efficiency, and adaptability in dynamic environments. The system architecture in DDS-based robotic systems enables seamless communication, distributed processing, and autonomous decision making.

Figure 10 illustrates middleware applications in robotic systems. Figure 10a shows how middleware integrates (or decouples [103]) different modules within a single robot, where the communication module allows the robot to interact with external systems through the middleware. Sensor modules, often physically connected via low-latency wiring, communicate with other modules or drivers in the control unit through middleware layers. Similarly, the control unit utilizes middleware to send commands to actuators, such as motors, pumps, fans, valves, or manipulator arms. Figure 10b presents a configuration where the robot communicates with external sensors and computers/servers through middleware. External sensors assist in environmental perception and planning, while servers log robotic activities or assist in complex computations, sharing the results with the robot through the middleware. Finally, Figure 10c demonstrates how middleware enables multiple robots to coordinate and exchange information to accomplish complex collaborative tasks. Thus, a robotic system architecture can be in one of these configurations individually or as a hybrid combination. Based on the reviewed literature, this paper further classifies DDS-based robotic system architectures into four key categories: Distributed Systems, Centralized Control, Decentralized Control, Real-Time Systems, and Heterogeneous Systems.




(**a**) Internal Integration (**b**) External Devices (**c**) Multi-Robot Configuration

**Figure 10.** Several configurations of using middleware in robotics.

#### 4.3.1. Distributed System Architecture

A distributed system in robotics refers to a network of multiple computing nodes, sensors, or robots that work collaboratively by sharing data and processing tasks in different locations. Unlike centralized systems, distributed robotics enables parallel processing, fault tolerance, and scalability, making it ideal for multi-robot coordination, industrial automation, and autonomous fleets [114]. One of its key advantages is redundancy, meaning that if one robot or node fails, others can compensate, ensuring system reliability. Furthermore, workload distribution improves efficiency by allowing multiple robots to process tasks simultaneously, reducing computational bottlenecks [114].

Figure 11 illustrates a typical distributed robotic system [88] featuring multiple robots equipped with sensors and motors. Poza-Luján et al. [88] developed the Frame Sensor Adapter to Control (FSACtrl), which is a distributed control architecture that leverages DDS for real-time communication. FSACtrl integrates Quality of Service (QoS) and Quality of Control (QoC) mechanisms to optimize mobile robot navigation. In this architecture, robots publish sensor data via DDS, and distributed control nodes subscribe to the data, process them, and generate left and right motor commands. These commands are then published and subsequently subscribed to by the robots to actuate movement within the simulation environment. The FSACtrl architecture enhances message handling, communication, and control efficiency, improving navigation performance.

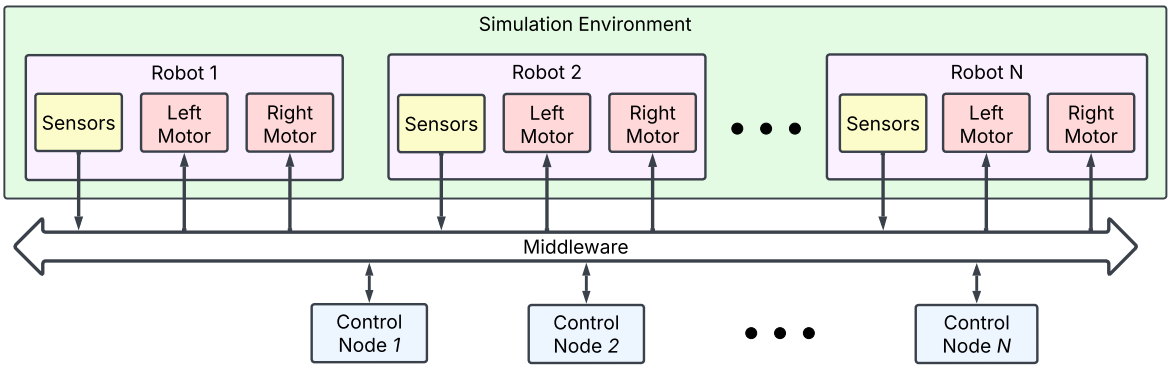



**Figure 11.** A distributed robotic system for mobile robots.

However, maintaining reliable performance in DDS-based distributed mobile robotic systems remains challenging due to the rigidity of fixed QoS attributes that struggle to adapt to fluctuating workloads and resource availability [60]. To address this, Inglés-Romero et al. [60] proposed using Nerve-based DDS middleware (see Service and Social Robots in Section 4.2), which automates DDS entity creation with user-defined QoS settings and employs a best-effort strategy for dynamic QoS policy adjustment.

Additionally, experiments show that developers must exercise caution when deploying DDS in distributed systems [115]. They show that deploying any middleware has

latency overhead, but one middleware has more latency than another. They also found a significant latency increase in all DDS middleware due to large payload sizes exceeding UDP fragmentation, hardware limitations (especially on resource-constrained devices like Raspberry Pi), OS energy-saving features, and internal processing delays within the DDS and ROS 2 systems.

#### 4.3.2. Centralized System Architecture

A centralized robotic system architecture relies on a single central controller or processing unit that manages the operations of all connected robots, sensors, and actuators. In this architecture, the central node is responsible for decision making, task allocation, and data processing, while individual robots act as execution units that follow commands from the controller. This model simplifies coordination and system management, making it ideal for applications requiring global awareness, strict task control, and structured operations, such as industrial automation, warehouse logistics, and surgical robotics. However, centralized architectures can suffer from scalability limitations, communication bottlenecks, and a single point of failure.

Despite these challenges, DDS enhances centralized systems by providing efficient data distribution, real-time communication, and security mechanisms, ensuring reliable connectivity between the central node and robotic units. Figure 12 shows the network architecture of ARP-ROS (See Data Management in Section 4.2.2) [96]. The distributed robots send their data to CEDDP for processing. The CEDDP then replies with control commands. The system shows lower CPU and memory usage for the robots, making it suitable for real-time distributed embedded systems such as low-cost multi-robot systems. The DDS helps manage the high volume of sensor data by ensuring efficient and reliable communication between the CEDDP and the robots, which is essential for the performance of multi-robot systems. It also allows for the seamless introduction of new nodes in the system with its dynamic discovery mechanism.

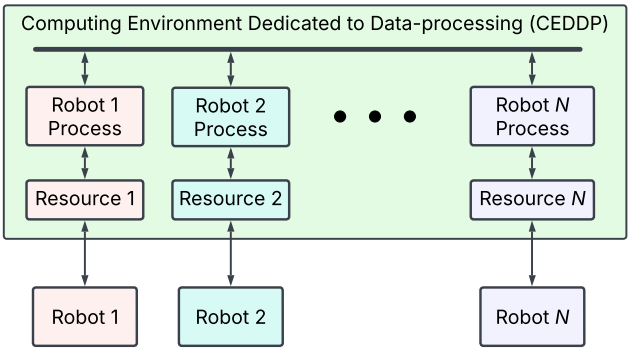



**Figure 12.** A distributed robot system with centralized computing support.

Another application of DDS-based centralized robotic systems is a UAV swarm controlled by QGroundControl v3.3.1 software as a Ground Control Station (GCS) [94]. The authors showed that the DDS security feature can protect a centralized robotic system. They used the Gazebo v9 robotics simulator to test three conditions: (1) ROS 1 alone, (2) a ROS 1–ROS 2 bridge without security, and (3) a ROS 1–ROS 2 bridge with security enabled. The findings showed that ROS 1 lacked built-in security, making UAVs vulnerable to rogue node attacks that could incapacitate, force-land, or alter flight paths. With the DDS security features of ROS 2 enabled, including authentication and encryption, the UAVs successfully blocked unauthorized access, preventing cyber attacks. However, the added security mechanisms introduced a latency overhead, increasing mission completion time by 16–17.5%. The study concludes that while ROS 2 improves UAV cybersecurity, its im-

plementation introduces performance tradeoffs that may affect real-time aerial operations. The researchers suspected excessive latency was due to the bridge but could not confirm it, because Gazebo v9 does not support ROS 2.

#### 4.3.3. Decentralized System Architecture

A decentralized robotic system architecture enables robots to operate autonomously without relying on a central controller, allowing for distributed decision making and task execution. In this architecture, each robot functions as an independent agent capable of communicating and coordinating with others through peer-to-peer interactions. This model is highly scalable, fault-tolerant, and adaptable, making it ideal for applications such as swarm robotics, search and rescue missions, and autonomous exploration. Unlike a centralized system, where a single controller dictates operations, a decentralized system eliminates the single point of failure, ensuring robustness in dynamic and unpredictable environments.

Table 6 compares centralized, decentralized, and distributed robotic systems, focusing on their control mechanisms, scalability, fault tolerance, communication overhead, decision-making speed, and the role of DDS. Centralized systems rely on a single controller for decision making, resulting in low scalability and fault tolerance, but offer fast initial decision making. However, they suffer from high communication overhead, as all data flows through the central entity. Decentralized systems, in contrast, allow fully autonomous operation, making them highly scalable and fault-tolerant, with low communication overhead and the fastest decision making. Distributed systems fall between these two models, offering shared control, moderate scalability and fault tolerance, and better decision-making speed than centralized systems. The role of DDS varies across these architectures: in centralized systems, DDS facilitates data distribution, ensuring efficient communication between robots and the central controller. In decentralized systems (see Figure 13), DDS enables dynamic discovery and peer-to-peer communication, allowing robots to interact autonomously. While in distributed systems, DDS supports real-time distributed communication, ensuring efficient data exchange while maintaining system coordination.

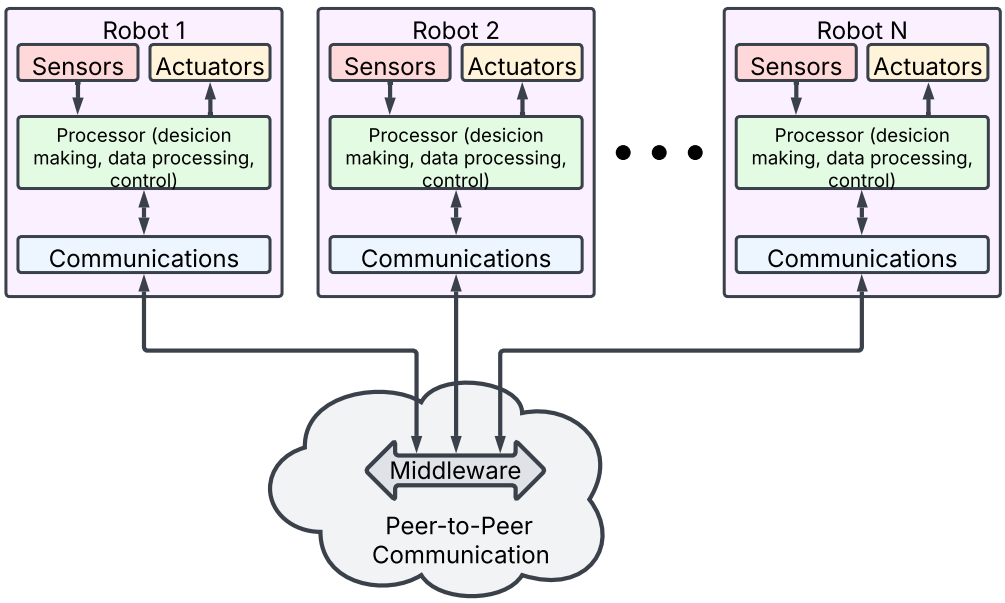



**Figure 13.** A decentralized robotic system architecture.

**Table 6.** Comparison of centralized, distributed, and decentralized robotic systems.

| Feature | Centralized System | Distributed System | Decentralized System |
|---|---|---|---|
| Control | Centralized decision making. | Shared control. | Fully autonomous operation. |
| Scalability | Low | Moderate to High | High |
| Fault Tolerance | Low | Moderate | High |
| Comm. Overhead | High | Moderate | Low |
| Decision Making | Fast | Faster | Fastest |
| Role of DDS | Facilitates centralized data distribution. | Enables distributed real-time communication. | Provides dynamic discovery and peer-to-peer communication. |

Essers and Vaneka [116] characterize traditional manufacturing systems as having a hierarchical structure and static configurations. The authors argue that centralized robotic systems in manufacturing face challenges that hinder their adaptability and performance in modern production settings, including reconfiguration complexity, vulnerability to single points of failure, rigid programming requirements, limited scalability, and inefficiency in dynamic environments. Thus, they proposed a decentralized robotic system architecture in which robots and human operators collaborate using distributed intelligence and DDS-based communication. Their system solves the four issues by enabling safe interactions, brand-independent programming, decentralized communication between robotic entities, reducing coordination complexity, and simplifying reconfigurations.

In another publication, Essers and Vaneker [61] presented a modular system architecture based on distributed intelligence and decentralized control, enabling the online reconfiguration of industrial robots within manufacturing facilities (see Task Management in Section 4.2.2). While both [61,116] leverage DDS in decentralized systems, ref. [116] places greater emphasis on adaptability for Small- and Medium-sized Enterprises (SMEs) through human–robot collaboration, whereas [61] focuses on performance optimization and modular reconfigurability. Together, these works demonstrate how decentralized robotic system architectures allow each component to make real-time decisions based on current data, enhancing efficiency, flexibility, and adaptability in manufacturing environments.

#### 4.3.4. Real-Time Systems

Real-time robotic systems require precise, time-sensitive communication and decision making to ensure predictable and responsive operations in dynamic environments. These systems rely on low-latency data exchange, deterministic task execution, and strict timing constraints, making them essential for applications such as autonomous vehicles, industrial automation, medical robotics, and swarm coordination. The effectiveness of real-time robotic systems depends on their ability to process sensor data, control commands, and establish system updates within defined time windows, preventing delays that could compromise safety or performance. Researchers often use DDS to synchronize the control of the robot with their simulation, ensuring that the real-time position and velocity of the robot match the planned trajectory [97].

The two most widely used middleware systems in real-time robotics applications are ROS 1 and ROS 2. Table 7 summarizes the middleware and architectures adopted in the reviewed publications from 2006 to 2024. The majority of papers (12 out of 41) utilized DDS middleware combined with ROS, reflecting the widespread integration of DDS into ROS-based robotic systems. In addition, four papers exclusively adopted DDS without ROS, while various specialized middleware solutions either improved on ROS or developed solutions without ROS.

**Table 7.** Middleware used by the reviewed publications.

| References | Middleware/Architecture | Count |
|---|---|---|
| [68,72,74,86,89,94,97,102,113,115,117,118] | DDS Middlewar + ROS | 12 |
| [61,70,116,119] | DDS | 4 |
| [103,120] | Nerve Middleware | 2 |
| [69,95] | Micro-ROS (DDS for Microcontrollers) | 2 |
| [58,121] | Callback-group-level Executor (Real-Time ROS2) | 2 |
| [110,122] | Secure ROS2 (SROS2 Security Layer) | 2 |
| [85] | fpgaDDS | 1 |
| [96] | ARP-ROS + CCA | 1 |
| [92] | RTI DDS | 1 |
| [123] | Dynamic DDS Binding | 1 |
| [36] | Fast DDS (Custom Lightweight use) | 1 |
| [78] | FogROS2 | 1 |
| [79] | FogROS2-SGC | 1 |
| [87] | embeddedRTPS | 1 |
| [98] | Kafka-ROS Bridge | 1 |
| [105] | Robotics Middleware Framework (RMF) | 1 |
| [88] | FSACtrl Architecture | 1 |
| [55] | Dynamic-DDS-RPC (Service Integration) | 1 |
| [124] | Context-Aware DDS for Aerospace Assembly | 1 |
| [90] | Simulated Interactive Robotics Initiative (SIRI) | 1 |
| [77] | Adaptive AUTOSAR DDS Communication | 1 |
| [109] | Synchronization Middleware for Co-simulation | 1 |
| [83] | Time-Sensitive Attribute Scheduler (ROS2-RTPS) | 1 |
| | Total | 41 |

Researchers in [118] evaluated the real-time performance of a ROS 2-based multi-agent service robot compared to ROS 1, conducting experiments in both idle and stressed environments. Periodic execution and jitter measurements on the Kobuki2 and Tetra DS-IV nodes demonstrated that ROS 2 maintained better real-time stability, with a maximum jitter of 2.5651 ms in idle and 2.9256 ms under stress, while ROS 1 exhibited higher latency and variability, making it less deterministic. Additionally, the timing inconsistencies of ROS 1 caused deviations from the planned Bezier curve trajectory, whereas ROS 2 successfully followed the reference path, maintaining accurate motion control. The findings confirm that ROS 2 provides superior real-time performance, stable execution under stress, and improved trajectory tracking, making it more reliable for real-time robotic applications.

The research shows a correlation between DDS configuration and its latency performance. Maruyama et al. [120] performed a general evaluation of DDS implementations in ROS 2, comparing it with OpenSplice [40] and RTI COnnext [37]. They found that DDS supports QoS policies, but there is a tradeoff between end-to-end latencies and throughput (specifically when message sizes exceed 64 KB), requiring packet division and increased processing time. OpenSplice performs better for local communication, utilizing multiple threads for faster processing, while Connext excels in remote communication, offering superior throughput.

DDS enhances ROS 2 with real-time capabilities, fault tolerance, and multi-platform compatibility, enabling communication across diverse systems without a master node. However, ROS 2 QoS policies currently lack sufficient real-time support. Also, Tobias et al. [115] studied the latency of ROS 2 in real-time distributed systems with default configuration and several DDS middlewares. Their work demonstrates that ROS 2 can lead to 50% more latency overhead than other low-level DDS communications, although the latencies highly depend on the DDS type.

Researchers are trying to improve the real-time performance of ROS 2 through configuration optimization or using better allocators or executors [58,89,121]: Yang and Azumi [58] replaced the standard ROS 2 rclcpp executor with a callback-group-level executor to improve real-time performance and reduce latency (44 µs with Cyclone DDS, 98 µs with Fast DDS). Teper et al. [89] later proposed constrained programming strategies that minimized end-to-end latencies by over 50%, enhancing system predictability and efficiency. Puck et al. [121] assess the real-time communication capabilities of ROS 2 due to the integration of DDS as middleware. The study evaluated real-time ROS 2 applications, including real-time safe dynamic memory allocation, process shielding, and network optimizations. The default ROS 2 allocator was found to be not suitable for real-time applications. However, using the TLSF allocator and preallocating messages could ensure bounded-time execution. Shielding real-time processes minimized disturbances and provides robust and predictable communication even under system load. The system latency was measured at under 422 µs across network boundaries and below 193 µs across thread boundaries, maintaining a 1 kHz control frequency. These findings confirm that proper system configurations can optimize the real-time performance of ROS 2.

#### 4.3.5. Heterogeneous Systems

Modern robotic systems often operate in heterogeneous environments, where robots, sensors, and computing platforms with different architectures, operating systems, and communication protocols must work seamlessly together. Heterogeneous robotic systems integrate various hardware and software components, enabling collaborative operations on diverse platforms. However, ensuring efficient communication, interoperability, and coordination in such systems presents significant challenges. DDS is crucial in addressing these challenges by providing a middleware framework that standardizes data exchange, supports dynamic discovery, and ensures real-time, scalable communication across diverse robotic entities. Heterogeneous robotic systems can achieve seamless integration, fault tolerance, and efficient resource management by leveraging the QoS policies of DDS, flexible data models, and multi-platform support. Thus, DDS is a vital enabler for multi-robot collaboration, industrial automation, and autonomous operations [77,109].

DDS supports heterogeneous robotic environments by abstracting the communication layer from hardware and operating system differences, enabling seamless interaction between devices with diverse architectures. Several studies have explored this capability: Parmar et al. [77] leveraged DDS to facilitate interoperability between ROS 2 and Adaptive AUTOSAR platforms; Emon et al. [109] customized DDS QoS settings to synchronize communication between heterogeneous robotic systems and legacy wireless network simulators; Calisi et al. [119] proposed DDS as a unifying interoperability layer for complex multi-agent systems. Furthermore, Fedi and Nasca [90] demonstrated how DDS facilitates integration between modern multi-robot fleets and legacy command and control infrastructures, maintaining compatibility while supporting autonomous and semi-autonomous collaboration. Collectively, these works highlight DDS's essential role in overcoming interoperability challenges and promoting flexible communication in heterogeneous robotic systems.

## 5. Challenges of Using DDS in Robotics

Integrating DDS into robotics offers numerous advantages, including platform independence, scalability, and real-time communication. However, implementing DDS in dynamic and complex robotic systems presents a significant challenge. Addressing these challenges is crucial to fully unlocking the potential of DDS in robotic systems. This section explores the key difficulties associated with incorporating DDS into robotics, focusing on four critical areas: performance and scalability, implementation and complexity, system

integration and compatibility, and security and robustness. By systematically examining these challenges, this review offers critical insights into the limitations of DDS in robotics, potentially identifying key gaps and highlighting priority areas for future research and development.

### 5.1. System Performance and Scalability

As robotics continues to adopt multi-robot systems, one of the most critical challenges is ensuring that DDS can scale efficiently and maintain seamless communication among a growing number of robots. This challenge involves managing dynamic interactions and diverse data flows without overwhelming limited system resources while maintaining real-time performance and reliability.

A notable issue is the accuracy of performance evaluation in robotic system settings, especially for ROS 2-based systems. Park et al. [118] note that scalability evaluations of ROS 2 may be misleading, because many tests occur under idle settings due to its heavy reliance on DDS.

Researchers have uncovered several sources of performance bottlenecks in robotic systems: Hartanto and Eich [74] found that as the number of robots increases, the available bandwidth per robot declines, resulting in communication inefficiencies. Sometimes, reliance on centralized computational elements, such as the CEDDP, can also lead to performance bottlenecks as robot numbers grow [96]. Another bottleneck arises from integrating hardware-mapped nodes with software components, which causes conflicting memory accesses when using delegate threads to call ROS functions [85].

Some researchers argue that the DDS middleware can contribute to performance degradation and scalability issues. Esser and Vaneker [61] identified two critical real-time issues with DDS in robotics: (1) despite DDS flexibility, it may not meet hard real-time requirements essential for timing-critical applications, and (2) varying the number of DDS participants can disrupt communication consistency, impacting system performance. These challenges underscore the need for dependable participant management and robust real-time scheduling.

Finally, scientists contend that integration challenges persist within and across the platforms of DDS-based robotic systems, hindering scalability in industrial robotic applications with diverse environmental conditions and heterogeneous platforms and hardware [97]. Managing DDS middleware to achieve real-time performance in diverse environments with different performance requirements is non-trivial [97]. They also observe that the lack of seamless integration with other platforms (including DDS to DDS integration) limits DDS applicability in broader robotics contexts [97,110].

### 5.2. Implementation and Complexity

Implementing DDS in robotic systems adds more complexity to the already present difficulties. Despite the ability of DDS to promote smooth communication and interoperability, configuration, deployment, and optimization complexities frequently prevent its widespread use in robotics. DDS adoption in robotics is greatly affected by the technical difficulties involved in its implementation, such as complex configuration procedures and the requirement for specialized knowledge.

Configuration challenges are a primary obstacle engineers and scientists encounter when deploying DDS in robotic systems. Robotic environments typically involve numerous heterogeneous devices, such as Bluetooth and Inertial Measurement Unit (IMU) sensors, which exacerbate the inherent complexity of DDS middleware [92]. Managing the modular architecture of DDS requires advanced configuration and coordination, and as the number of devices and data streams increases, communication overhead can degrade real-time

performance. Furthermore, scalability intensifies system complexity, demanding more effective strategies to support growing and dynamic networks. Thus, there is a need for more user-friendly configuration processes.

Some middleware require QoS settings as part of their configuration rituals. DDS middleware is notorious for having a large number of QoS parameters. Users unfamiliar with middleware technologies may become overwhelmed by the various QoS settings and configurations [70]. This high learning curve and a lack of real-world case studies demonstrating the usefulness of DDS in diverse robotic contexts prevent DDS from being widely adopted and made accessible in the field. Other DDS middleware types have few QoS parameters, which may not cover the range of real-world situations encountered in robotics applications beyond the controlled testing environment [109].

Time synchronization is a critical flaw in DDS-based robotic systems, which is exacerbated by increased packet delays in system components and the inherent unreliability and overhead of computer-to-computer communication [55,109]. Tight coupling between the middleware framework and service implementation further complicates synchronization [55]. These challenges affect the accuracy of service response time measurements and hinder accurate, consistent coordination between robots, ultimately impairing the performance and reliability of real-time communication.

Studies show that standalone DDS systems, like FastDDS, perform better than DDS systems in ROS in time-sensitive communication [105]. Thus, some researchers proposed a mechanism that enables dynamic binding of various DDS implementations based on communication characteristics like data size, reliability, and subscriber location to optimize internode communication in ROS 2 [123]. However, this method makes the middleware even more complex and necessitates careful consideration of some parameters during implementation. Furthermore, the message format conversions brought about by the integration of ROS 2 with Fast DDS increase latency and make it less appropriate for real-time applications.

### *5.3. System Integration and Compatibility*

Compatibility and integration in robotic systems are complicated, especially in settings with various heterogeneous components. Robotic platforms often require seamless communication across complex middleware, diverse software frameworks, and specialized hardware architectures. An effective deployment of DDS middleware relies on ensuring reliable interoperability across these layers. Integration difficulties become even more pronounced when combining well-established frameworks like ROS with various platforms, and communication protocols that may not fully align with DDS standards. Addressing these compatibility challenges is critical to enabling DDS adoption in real-world robotic applications.

Interoperability issues are particularly evident in multi-robot systems, where efficient coordination depends heavily on smooth communication. Communication inconsistencies can cause misalignment among collaborative robots, resulting in frequent task failures [105]. Vendor dependencies further aggravate interoperability challenges, leading to high integration and maintenance costs [119]. This survey (see Cross-Platform Communication in Section 4.2.4) has discussed cross-vendor interoperability issues between DDS implementations [110], leading to high upgrade costs or avoiding implementation altogether.

Hardware compatibility is another challenge, especially when deploying DDS in actual robotic systems. It is noteworthy because single-hop network simulations are inadequate to analyze the complicated multi-hop networks in real-world deployment, which is too expensive or impossible to duplicate in a lab setting [83]. Additionally, hardware compatibility issues can lead to poor energy, latency, and reliability performance [115].

Software compatibility is also as bad as its hardware counterpart, further hindering DDS adoption. Software incompatibilities may lead to competing memory access, causing bottlenecks, which can impair the whole performance of the system [85]. Additionally, incompatibilities between DDS middleware and proprietary manufacturing software complicate integration [116]. Software integration is another problem. For example, ROS 2 has weak task management features, which can severely limit its use in intricate, multi-threaded robotic systems [102].

Researchers argue that standardization efforts could significantly alleviate the integration and compatibility challenges associated with DDS deployment in robotic systems [119]. The lack of unified techniques and specifications complicates robotic system development and integration, creating barriers to interoperability across different platforms.

### 5.4. Robustness

Robustness is the extent to which a system or component can perform its intended functions, even when subjected to invalid data or stressful environmental factors. In the context of DDS middleware, robustness refers to the system's ability to maintain reliable communication and fault tolerance despite encountering invalid or exceptional inputs, ensuring consistent performance and stability in critical applications [125]. Researchers commonly evaluate middleware robustness using the crash scale, which classifies system responses into Hindering, Silent, Abort, Restart, and Catastrophic categories [126]. The robustness of DDS middleware is crucial, since it forms the foundation of a robotic system, ensuring real-time communication between different parts of the robotic ecosystem.

Effective fault recovery requires efficient communication, which can be challenging due to bandwidth limitations and network uncertainties in wireless environments. To prevent deadlock situations when robots lose connection, Jeong et al. [113] used a dynamic discovery mechanism in the DDS to allow robots to enter or exit the DDS domains dynamically. However, developers must ensure their DDS middleware can regain performance after a participant enters or exits the domain [61].

Proper DDS configurations, like QoS settings, guarantee message transport, further reducing the likelihood of failure during data exchange between entities [98,116]. Essers and Vaneker [61] recommended using integrated approaches to implement the DDS API to yield smaller message latencies compared to client/server connections, which are often less reliable. This reliability is essential for maintaining robust communication in manufacturing environments. Lastly, the system architecture can help a DDS-based robotic system achieve high levels of robustness. Researchers recommend a distributed control robotic system or decentralized robotic architecture to ensure overall system robustness [61,116].

## 6. Security and Privacy Implications of DDS in Robotics

Section 5 examines the challenges of DDS middleware in robotic system communications, focusing on key limitations and related features. However, in the context of robotic systems, security and privacy concerns are of utmost importance, as robots often operate in sensitive environments and handle mission-critical data that could be vulnerable to cyber threats. Ensuring that the confidentiality, integrity, and availability (CIA) of data transmitted over DDS are maintained allows for robust and resilient robotic operations. This section explores the security challenges identified in the reviewed literature, analyzing potential vulnerabilities, attack surfaces, and mitigation strategies.

### 6.1. Authentication and Authorization

Authentication and authorization are the building blocks for protecting DDS communications, specifically in robotics, where multiple components (i.e., sensor, actuator, and

control node) must work through secure channels. Sergio and Preetha [94] emphasize the importance of these features in unmanned military-grade systems, where DDS ensures secure real-time communication between the ground control station and UAVs. They recommend using encryption and authentication to prevent unauthorized data injection and protect UAV swarms from cyber threats. Similarly, Mayank et al. [105] demonstrate the integration of authentication and authorization mechanisms in the RMF, ensuring secure communication between robot fleets and improving system robustness in real-world operations. However, vulnerabilities may persist. Thus, these features must be effectively implemented through appropriate mechanisms to ensure that only authorized access to robotic systems is granted to authenticated robotic nodes.

### 6.2. Data Protection

In DDS-enabled robotic systems, data protection focuses on safeguarding sensitive information from unauthorized access, ensuring its CIA. Several studies emphasize the importance of encryption and secure data exchange in robotic networks based on DDS middleware.

For example, Kaiyuan et al. [79] describe the use of cryptography and Datagram Transport Layer Security (DTLS) in FogROS2-SGC, keeping messages confidential and untampered, ensuring that they are from a legitimate sender. This method enhances access control at the topic level to restrict specific ROS 2 topics for unauthorized nodes, thereby securing robotic networks against external intruders. The system design minimizes attack surfaces and is essential for globally distributed robotic applications.

Also, Fernandez et al. [68] show that advanced algorithms like Elliptic Curve Cryptography (ECC) are used to secure complete RTPS messages in multi-unmanned system networks. At the same time, security features often come with performance inefficiencies in the form of latency and throughput overhead, highlighting the tradeoff between data protection and potential operational bottlenecks.

Enforcing strict QoS parameters through secure communication protocols is an important data protection strategy. Daniele et al. [119] describe how QoS establishes the predictable and reliable data exchange necessary to maintain the confidentiality and integrity of sensitive mission-critical information in networked robotic systems.

### 6.3. System Architecture and Protocols

Recent DDS middleware frameworks contain a security layer. An illustration of this is the shift from ROS 1 to ROS 2, which emphasizes systems security due to the usage of DDS middleware.

Secure ROS 2 (SROS 2) is a security module for ROS 2 that lays down core security capabilities like authentication, encryption, and access control. However, Gelei et al. [122] uncovered serious weaknesses in SROS 2, such as bypassed access control and intercepting confidential messages. To solve this problem, advanced encryption techniques, such as private broadcast encryption, were proposed to ensure secure communications in robotic systems.

Meanwhile, security-enhanced robotics frameworks like the RMF significantly enhance system architecture by integrating secure communication and access control mechanisms. Mayank and Kamalanathan [105] highlight DDS security tools in RMF that allow access control and communication encryption for robot fleets. The framework combines system architecture with secure communication protocols to fulfill the requirements of dynamic and mission-critical robotic environments.

### 6.4. Other Security Challenges

There are additional security constraints in robotics compared to typical DDS middleware applications. Performance tradeoffs associated with enabling security features, such as authentication and encryption, are well documented in the literature: Aartsen et al. [110] pointed out that enabling these features in DDS middleware can significantly increase system complexity and latency, posing challenges for applications that require real-time communication.

Furthermore, scalability and interoperability challenges in large robotic networks are further exacerbated by security overheads. To address these issues, Kaiyuan et al. [79] developed FogROS2-SGC, which is a platform that facilitates secure communication between heterogeneous ROS 2 networks operating across distinct environments. However, its design incorporates complex cryptographic and access control subsystems, which increase system complexity and computational overhead.

Emerging approaches such as blockchain integration and differential privacy have been proposed to enhance security through secure access logging and anonymous data handling. Cardenas et al. [117] discuss these advanced techniques, which, while offering significant security benefits, impose considerable computational demands. Consequently, it is crucial to carefully balance security enhancements with the resource constraints inherent to robotic systems during system design.

## 7. Conclusions

This paper explored the applications, challenges, and critical privacy and security considerations of employing DDS in robotic systems. Using the PRISMA systematic literature review framework, we examined 41 state-of-the-art publications and used them to provide a comprehensive analysis of the field. Our findings reveal that DDS applications in robotics can be categorized according to communication types or models, specific application domains, and system architectures. Emerging technologies like fog computing and cloud–edge–end fusion leverage DDS middleware to enhance performance and secure communication channels. The role of DDS in robotics varies widely, depending on the application domain, the management goals engineers aim to achieve, and the nature of the robotic systems involved. Architecturally, DDS can support various types of system integration or enable specific architectural frameworks tailored to robotic environments. However, we identified several significant challenges in using DDS for robotics. These include performance and scalability limitations, the complexity of integrating heterogeneous and distributed robotic systems, compatibility issues, and robustness and interoperability difficulties. Such challenges highlight the need for continuous development and optimization to maximize the potential of DDS in this field. Similarly, security and privacy remain critical concerns in DDS-based robotics. Although DDS has notable security features, vulnerabilities persist. For example, the weaknesses in SROS 2 expose authentication and authorization gaps. Data protection issues present risks for mission-critical applications, and enabling advanced security features incurs overhead. Performance tradeoffs can impact scalability and interoperability, further complicating the deployment of secure, efficient robotic systems. While DDS has shown great promise in enabling advanced robotic applications, addressing its challenges and strengthening its security and privacy measures will be crucial to widespread adoption in increasingly complex and mission-critical robotic environments.

**Author Contributions:** Conceptualization, B.A., M.L.G., and A.D.; methodology, F.A.; validation, F.A., M.L.G., A.D., and A.A.; formal analysis, F.A., M.L.G., A.D., and A.A.; investigation, F.A., M.L.G., and A.D.; resources, B.A., F.A., M.L.G., A.D., and E.A.-N.; data curation, M.L.G. and A.D.; writing—original draft preparation, M.L.G. and A.D.; writing—review and editing, F.A., A.A., and E.A.-N.; visualization, F.A., M.L.G., and A.D.; supervision, B.A.; project administration, B.A.; funding acquisition, B.A. All authors have read and agreed to the published version of the manuscript.

**Funding:** The APC was funded by the Deanship of Research Oversight and Coordination (DROC), King Fahd University of Petroleum and Minerals (KFUPM).

**Acknowledgments:** The authors acknowledge all support provided by Alfozan Academy, the Center of Excellence in Development of Nonprofit Organizations (CEDNPO), and King Fahd University of Petroleum and Minerals (KFUPM).

**Conflicts of Interest:** The authors declare no conflicts of interest.

## Abbreviations

The following abbreviations are used in this manuscript:

| | |
|---|---|
| AGV | Automated Guided Vehicle |
| API | Application Programming Interface |
| ARP-ROS | Aggregated Robot Processing-Robot Operating System |
| AXIS | Advanced eXtensible Interface-streaming |
| BE | Best Effort |
| BTs | Behavior Trees |
| C2 | Command and Control |
| CA | Certificate Authority |
| CBG-Executor | Callback-Group-Level Executor |
| CCA | Cache-Control Algorithm |
| CEDDP | Computing Environment Dedicated to Data Processing |
| CIA | Confidentiality, Integrity, and Availability |
| CPU | Central Processing Unit |
| DDS | Data Distribution Service |
| DIS | Distributed Interactive Simulation |
| DRE | Distributed Real-Time Embedded |
| DTLS | Datagram Transport Layer Security |
| ECC | Elliptic Curve Cryptography |
| FSACtrl | Frame Sensor Adapter to Control |
| GCS | Ground Control Station |
| HLA | High-Level Architecture |
| HMTs | Hardware-Mapped Topics |
| IEC | International Electrotechnical Commission |
| IoT | Internet of Things |
| IPCs | Industrial Personal Computers |
| M2M | Machine-to-Machine |
| MACs | Message Authentication Codes |
| MAVROS | MAVLink Extending to ROS |
| MMS | Mission Management Station |
| MRS | Multi-Robot System |
| NIEM | National Information Exchange Model |
| OMG | Object Management Group |
| PLCs | Programmable Logic Controllers |
| QoC | Quality of Control |
| QoS | Quality of Service |
| RAS | Robotic Automation System |

| | |
|---|---|
| RLbI | Robot Learning by Imitation |
| RMF | Robotics Middleware Framework |
| ROS | Robot Operating System |
| ROS 2 | Robot Operating System 2 |
| RQs | Research Questions |
| RTI | Real-Time Innovation |
| RTOS | Real-Time Operating System |
| RTPS | Real-Time Publish–Subscribe |
| SIRI | Simulated Interactive Robotics Initiative |
| SLAM | Simultaneous Localization and Mapping |
| SLR | Systematic Literature Review |
| SMEs | Small- and Medium-sized Enterprises |
| SO | Soft Real-Time |
| SSL | Secure Sockets Layer |
| SSR | SeoulTech Service Robot |
| TCP | Transmission Control Protocol |
| TLSF | Two-Level Segregated Fit |
| TSN | Time-Sensitive Networking |
| UAVs | Unmanned Aerial Vehicles |
| UDP | User Datagram Protocol |
| UGVs | Unmanned Ground Vehicles |
| XML | Extensible Markup Language |

## References


1. Koditschek, D.E. What Is Robotics? Why Do We Need It and How Can We Get It? *Annu. Rev. Control. Robot. Auton. Syst.* **2021**, *4*, 1–33. [CrossRef]
2. Siciliano, B.; Khatib, O. (Eds.) *Springer Handbook of Robotics*; Springer Handbooks; Springer International Publishing: Cham, Switzerland, 2016. [CrossRef]
3. Craig, J.J. *Introduction to Robotics: Mechanics and Control*, 3rd ed.; Pearson Prentice Hall: Upper Saddle River, NJ, USA, 2005.
4. Bekey, G.A. *Autonomous Robots: From Biological Inspiration to Implementation and Control*; MIT Press: Cambridge, MA, USA, 2005.
5. Garcia, E.; Jimenez, M.A.; Santos, P.G.D.; Armada, M. The evolution of robotics research. *IEEE Robot. Autom. Mag.* **2007**, *14*, 90–103. [CrossRef]
6. Ben-Ari, M.; Mondada, F. *Robots and Their Applications*; Springer International Publishing: Cham, Switzerland, 2018; pp. 1–20. [CrossRef]
7. Baumann, C.; Martinoli, A. A modular functional framework for the design and evaluation of multi-robot navigation. *Robot. Auton. Syst.* **2021**, *144*, 103849. [CrossRef]
8. Selcuk, A.A. A Guide for Systematic Reviews: PRISMA. *Turk. Arch. Otorhinolaryngol.* **2019**, *57*, 57–58. [CrossRef]
9. Rathi, N.; Bandavade, P.; Chaubey, H.; Jaju, S.; Thakur, S. Comprehensive Study of Advantages, Limitations, Applications and Similar Tool of ChatGPT. In Proceedings of the 2024 International Conference on Artificial Intelligence and Quantum Computation-Based Sensor Application (ICAIQSA), Nagpur, India, 20–21 December 2024; pp. 1–4. [CrossRef]
10. Fitria, T.N. Grammarly as AI-powered English writing assistant: Students' alternative for writing English. *Metathesis J. Engl. Lang. Lit. Teach.* **2021**, *5*, 65–78. [CrossRef]
11. Bishop, T.A.; Karne, R.K. A survey of middleware. In Proceedings of the CATA, Honolulu, HI, USA, 26–28 March 2003; pp. 254–258.
12. Almadani, B.; Alissa, S.; Alshareef, R.; Aliyu, F.; Al-Nahari, E. A Survey of Middleware Adoption in Nonprofit Sectors: A Sustainable Development Perspective. *Sustainability* **2024**, *16*, 8904. [CrossRef]
13. Sadok, D.F.H.; Gomes, L.L.; Eisenhauer, M.; Kelner, J. A middleware for industry. *Comput. Ind.* **2015**, *71*, 58–76. [CrossRef]
14. Susnjara, S.; Smalley, I. What Is Middleware? 2024. Available online: https://www.ibm.com/think/topics/middleware (accessed on 27 November 2024).
15. Vinoski, S. Where is middleware. *IEEE Internet Comput.* **2002**, *6*, 83–85. [CrossRef]
16. Jiang, Y.; Liu, Q.; Qin, C.; Su, J.; Liu, Q. Message-oriented Middleware: A Review. In Proceedings of the 2019 5th International Conference on Big Data Computing and Communications (BIGCOM), Qingdao, China, 9–11 August 2019; pp. 88–97. [CrossRef]
17. Abraham, B.; Aguilar, J.; Leiss, E. Middleware for Improving Security in a Component Based Software Architecture. *Netw. Comput. Adv. Inf. Manag.* **2008**, *1*, 502–509. [CrossRef]

18. Cahill, V. Building a dynamically reconfigurable minimum CORBA platform with components, connectors and language-level support. In Proceedings of the Middleware 2000 Workshop on Reflective Middleware (RM 2000), New York, NY, USA, 7–8 April 2000; pp. 1–2.
19. Qureshi, N.A.; Perini, A. An Agent-Based Middleware for Adaptive Systems. In Proceedings of the Eighth International Conference on Quality Software, Oxford, UK, 12–13 August 2008; pp. 423–428. [CrossRef]
20. Hapner, M.; Burridge, R.; Sharma, R.; Fialli, J.; Stout, K. *Java Message Service*; Sun Microsystems Inc.: Santa Clara, CA, USA, 2002; Volume 9. Available online: https://raw.githubusercontent.com/TIM168/technical_books/master/Java/JMS20.pdf (accessed on 27 November 2024).
21. Mackey, A., Windows Communication Foundation. In *Introducing .NET 4.0: With Visual Studio 2010*; Apress: Berkeley, CA, USA, 2010; pp. 159–173. [CrossRef]
22. RTI. Connext Product Suite for Intelligent Distributed Systems. 2025. Available online: https://www.rti.com/products (accessed on 24 February 2025).
23. eProsima. eProsima Fast DDS Documentation—Fast DDS 3.1.2 Documentation. 2025. Available online: https://fast-dds.docs.eprosima.com/ (accessed on 24 February 2025).
24. OMG. OMG Data Distribution Service (DDS) Version 1.4. 2015. Available online: http://www.omg.org/spec/DDS/1.4 (accessed on 27 November 2024).
25. Newton-King, J.; Latham, L.; Pickett, W.; Anderson, R. Why migrate WCF to asp.net core grpc — Microsoft Learn, 2024. Available online: https://bit.ly/3CLlwFb (accessed on 27 November 2024).
26. Boonma, P.; Suzuki, J. TinyDDS: An interoperable and configurable publish/subscribe middleware for wireless sensor networks. In *Wireless Technologies: Concepts, Methodologies, Tools and Applications*; IGI Global: Hershey, PA, USA, 2012; pp. 819–846.
27. Almadani, B.; Saeed, B.; Alroubaiy, A. Healthcare systems integration using Real Time Publish Subscribe (RTPS) middleware. *Comput. Electr. Eng.* **2016**, *50*, 67–78. [CrossRef]
28. Fersi, G. Study of Middleware for Internet of Healthcare Things and Their Applications. *Impact Digit. Technol. Public Health Dev. Dev. Ctries.* **2020**, *12157*, 223–231.
29. Ahn, M.K.; Park, Y.W.; Jee, T.Y. Component based open middleware architecture for autonomous navigation system. In Proceedings of the Unmanned Systems Technology IX, SPIE, Orlando, FL, USA, May 2007; pp. 653–660. [CrossRef]
30. Mohamed, N.; Al-Jaroodi, J.; Jawhar, I.; Lazarova-Molnar, S. Middleware requirements for collaborative unmanned aerial vehicles. In Proceedings of the 2013 International Conference on Unmanned Aircraft Systems (ICUAS), Atlanta, GA, USA, 28–31 May 2013; pp. 1051–1060. [CrossRef]
31. Bode, V.; Trinitis, C.; Schulz, M.; Buettner, D.; Preclik, T. DDS Implementations as Real-Time Middleware—A Systematic Evaluation. In Proceedings of the 2023 IEEE 29th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), Niigata, Japan, 30 August–1 September 2023; pp. 186–195. [CrossRef]
32. Köksal, Ö.; Tekinerdogan, B. Obstacles in Data Distribution Service Middleware: A Systematic Review. *Future Gener. Comput. Syst.* **2017**, *68*, 191–210. [CrossRef]
33. Palade, A.; Cabrera, C.; Li, F.; White, G.; Razzaque, M.A.; Clarke, S. Middleware for Internet of Things: An evaluation in a small-scale IoT environment. *J. Reliab. Intell. Environ.* **2018**, *4*, 3–23. [CrossRef]
34. Choi, H.Y.; King, A.L.; Lee, I. Making DDS really real-time with OpenFlow. In Proceedings of the 2016 International Conference on Embedded Software (EMSOFT), Pittsburgh, PA, USA, 2–7 October 2016; pp. 1–10. [CrossRef]
35. Profanter, S.; Tekat, A.; Dorofeev, K.; Rickert, M.; Knoll, A. OPC UA versus ROS, DDS, and MQTT: Performance Evaluation of Industry 4.0 Protocols. In Proceedings of the 2019 IEEE International Conference on Industrial Technology (ICIT), Pittsburgh, PA, USA, 1–7 October 2016; pp. 955–962. [CrossRef]
36. Appius, A.X.; Bauer, E.; Blöchlinger, M.; Kalra, A.; Oberson, R.; Raayatsanati, A.; Strauch, P.; Suresh, S.; von Salis, M.; Katzschmann, R.K. RAPTOR: Rapid Aerial Pickup and Transport of Objects by Robots. In Proceedings of the 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Kyoto, Japan, 23–27 October 2022; pp. 349–355. [CrossRef]
37. RTI. RTI Connext DDS. 2024. Available online: https://www.rti.com/ (accessed on 24 February 2025).
38. Object Computing. OpenDDS. 2025. Available online: https://opendds.org/ (accessed on 24 February 2025).
39. Twin Oaks Computing. CoreDX DDS Data Distribution Service Middleware | Twin Oaks Computing, Inc. 2020. Available online: https://www.twinoakscomputing.com/coredx/ (accessed on 24 February 2025).
40. ADLINK Technology. Vortex OpenSplice. 2021 Available online: https://www.adlinktech.com/en/vortex-opensplice-data-distribution-service (accessed on 24 February 2025).
41. GurumNetworks. GurumDDS. 2025. Available online: https://www.gurum.cc/gurumdds_eng (accessed on 24 February 2025).
42. Eclipse Incubation. Eclipse Cyclone DDS—Home. 2024. Available online: https://cyclonedds.io/ (accessed on 24 February 2025).
43. Kim, H.; Kim, D.K.; Alaerjan, A. ABAC-Based Security Model for DDS. *IEEE Trans. Dependable Secur. Comput.* **2022**, *19*, 3113–3124. [CrossRef]

44. Nwadiugwu, W.; Kim, D.; Ejaz, W.; Anpalagan, A. MAD-DDS: Memory-efficient automatic discovery data distribution service for large-scale distributed control network. *IET Commun.* **2023**, *17*, 1432–1446. [CrossRef]
45. Song, B.; Hu, X.; Xiao, J.; Zhang, G.; Wang, S.; Zhou, Q. Integration of Data Distribution Service into Partitioned Real-time Embedded Systems. In Proceedings of the 2020 15th IEEE Conference on Industrial Electronics and Applications (ICIEA), Virtual Event, 9–13 November 2020; pp. 1606–1611. [CrossRef]
46. Alaerjan, A.; Kim, D.K.; Ming, H.; Kim, H. Configurable DDS as Uniform Middleware for Data Communication in Smart Grids. *Energies* **2020**, *13*, 1839. [CrossRef]
47. Wen, J.; Jiang, L.; Zhu, J.; Qiu, Z.; Chu, C.C. Design of Data Distributed Service-Based Distributed Co-Simulation Platform of Power Systems. In Proceedings of the 2023 IEEE Industry Applications Society Annual Meeting (IAS), Nashville, TN, USA, 29 October–2 November 2023; pp. 1–7. [CrossRef]
48. Zhang, J.; Yu, X.; Sier, H.; Queralta, J.P.; Westerlund, T. Comparison of DDS, MQTT, and Zenoh in Edge-to-Edge and Edge-to-Cloud Communication for Distributed ROS 2 Systems. *arXiv* **2023**, arXiv:2309.07496. Available online: https://api.semanticscholar.org/CorpusID:261823232 (accessed on 27 November 2024).
49. Du, J.; Gao, C.; Feng, T. Formal Safety Assessment and Improvement of DDS Protocol for Industrial Data Distribution Service. *Future Internet* **2023**, *15*, 24. [CrossRef]
50. Alaerjan, A.; Kim, D.K. Adopting DDS to Smart Grids: Towards Reliable Data Communication. In *Smart Cities, Green Technologies, and Intelligent Transport Systems*; Helfert, M.; Klein, C.; Donnellan, B.; Gusikhin, O., Eds.; Springer: Cham, Switzerland, 2017; pp. 154–169.
51. RFWW. DDS Protocol Architecture Basics: DDS Protocol in IoT. 2024. Available online: https://www.rfwireless-world.com/Terminology/DDS-protocol-architecture.html (accessed on 1 March 2025).
52. ADLINK Technology . *Using DDS for Scalable, High Performance, Real-time Data Sharing in Next Generation Modeling & Simulation Systems*; Whitepaper; ADLINK Technology Inc.: Taoyuan, Taiwan, 2019.
53. DDSF. What's in the DDS Standard? 2024. Available online: https://www.dds-foundation.org/omg-dds-standard/ (accessed on 1 March 2025).
54. Bellavista, P.; Corradi, A.; Foschini, L.; Pernafini, A. Data Distribution Service (DDS): A performance comparison of OpenSplice and RTI implementations. In Proceedings of the 2013 IEEE Symposium on Computers and Communications (ISCC), Split, Croatia, 7–10 July 2013; pp. 377–383. [CrossRef]
55. Zhang, X.; Zhang, X.; Wang, S.; Xiao, P. Design and Implementation of Robot Middleware Service Integration Framework Based on DDS. In Proceedings of the 2022 IEEE International Conference on Real-time Computing and Robotics (RCAR), Guiyang, China, 17–22 July 2022; pp. 588–593. [CrossRef]
56. OMG. The Real-Time Publish-Subscribe Protocol DDS Interoperability Wire Protocol (DDSI-RTPS$^{TM}$) Specification. 2022. Available online: https://www.omg.org/spec/DDSI-RTPS/2.5/PDF (accessed on 1 March 2025).
57. Object Management Group (OMG). DDS Security Specification Version 1.1, 2024. Available online: https://www.omg.org/spec/DDS-SECURITY (accessed on 27 November 2024).
58. Yang, Y.; Azumi, T. Exploring Real-Time Executor on ROS 2. In Proceedings of the 2020 IEEE International Conference on Embedded Software and Systems (ICESS), Shanghai, China, 10–11 December 2020; pp. 1–8. [CrossRef]
59. Al-hammouri, M.; Madani, B.; Aloqaily, M.; Ridhawi, I.A.; Jararweh, Y. Scalable Video Streaming for Real-Time Multimedia Applications over DDS Middleware for Future Internet Architecture. In Proceedings of the 2018 IEEE/ACS 15th International Conference on Computer Systems and Applications (AICCSA), Aqaba, Jordan, 28 October–1 November 2018; pp. 1–6. [CrossRef]
60. Inglús-Romero, J.F.; Romero-Garcés, A.; Vicente-Chicote, C.; Martínez, J. A Model-Driven Approach to Enable Adaptive QoS in DDS-Based Middleware. *IEEE Trans. Emerg. Top. Comput. Intell.* **2017**, *1*, 176–187. [CrossRef]
61. Essers, M.; Vaneker, T. Evaluating a Prototype Approach to Validating a DDS-based System Architecture for Automated Manufacturing Environments. *Procedia CIRP* **2014**, *25*, 385–392.
62. White, T.; Johnstone, M.N.; Peacock, M. An investigation into some security issues in the DDS messaging protocol. In Proceedings of the 15th Australian Information Security Management Conference, Perth, Australia, 5–6 December 2017.
63. Beckman, K.; Reininger, J. Adaptation of the DDS Security Standard for Resource-Constrained Sensor Networks. In Proceedings of the 2018 IEEE 13th International Symposium on Industrial Embedded Systems (SIES), Graz, Austria, 6–8 June 2018; pp. 1–4. [CrossRef]
64. Wagner, P.G.; Birnstill, P.; Beyerer, J. DDS Security+: Enhancing the Data Distribution Service With TPM-based Remote Attestation. In Proceedings of the 19th International Conference on Availability, Reliability and Security—ARES '24, Vienna, Austria, 30 July–2 August 2024. [CrossRef]
65. Han, J.H. Message encryption methods for DDS security performance improvement. *J. Korea Inst. Inf. Commun. Eng.* **2018**, *22*, 1554–1561.
66. Friesen, M.; Karthikeyan, G.; Heiss, S.; Wisniewski, L.; Trsek, H. A comparative evaluation of security mechanisms in DDS, TLS and DTLS. In *Kommunikation und Bildverarbeitung in der Automation*; Springer: Berlin/Heidelberg, Germany, 2020; pp. 201–216.

67. Luo, Q.; Shi, R. A Decentralized Access Control Framework For DDS. In Proceedings of the 2021 IEEE Sixth International Conference on Data Science in Cyberspace (DSC), Shenzhen, China, 9–11 October 2021; pp. 211–218. [CrossRef]
68. Fernandez, J.; Allen, B.; Thulasiraman, P.; Bingham, B. Performance Study of the Robot Operating System 2 with QoS and Cyber Security Settings. In Proceedings of the 2020 IEEE International Systems Conference (SysCon), Montreal, QC, Canada, 20–23 April 2020; pp. 1–6. [CrossRef]
69. Mamani-Saico, A.; Yanyachi, P.R. Implementation and Performance Study of the Micro-ROS/ROS2 Framework to Algorithm Design for Attitude Determination and Control System. *IEEE Access* **2023**, *11*, 128451–128460. [CrossRef]
70. Calvo, I.; Pérez, F.; Etxeberria-Agiriano, I.; de Albéniz, O.G. Designing High Performance Factory Automation Applications on Top of DDS. *Int. J. Adv. Robot. Syst.* **2013**, *10*, 205. [CrossRef]
71. Aliyu, F.; Umar, S.; Al-Duwaish, H. A Survey of Applications of Artificial Neural Networks in Wireless Sensor Networks. In Proceedings of the 2019 8th International Conference on Modeling Simulation and Applied Optimization (ICMSAO), Manama, Bahrain, 15–17 April 2019; pp. 1–5. [CrossRef]
72. Jeong, S.; Ga, T.; Jeong, I.; Choi, J. Behavior tree driven multi-mobile robots via data distribution service (DDS). In Proceedings of the 2021 21st International Conference on Control, Automation and Systems (ICCAS), Jeju, Korea, 12–15 October 2021; pp. 1633–1638.
73. Iovino, M.; Scukins, E.; Styrud, J.; Ögren, P.; Smith, C. A survey of Behavior Trees in robotics and AI. *Robot. Auton. Syst.* **2022**, *154*, 104096. [CrossRef]
74. Hartanto, R.; Eich, M. Reliable, cloud-based communication for multi-robot systems. In Proceedings of the 2014 IEEE International Conference on Technologies for Practical Robot Applications (TePRA), Woburn, WA, USA, 14–15 April 2014; pp. 1–8. [CrossRef]
75. Stojmenovic, I.; Wen, S.; Huang, X.; Luan, H. An overview of fog computing and its security issues. *Concurr. Comput. Pract. Exp.* **2016**, *28*, 2991–3005. [CrossRef]
76. Das, R.; Inuwa, M.M. A review on fog computing: Issues, characteristics, challenges, and potential applications. *Telemat. Inform. Rep.* **2023**, *10*, 100049. [CrossRef]
77. Parmar, N.; Ranga, V.; Simhachalam Naidu, B. Syntactic Interoperability in Real-Time Systems, ROS 2, and Adaptive AUTOSAR Using Data Distribution Services: An Approach. In *Inventive Communication and Computational Technologies*; Springer: Singapore, 2020; pp. 257–274. [CrossRef]
78. Ichnowski, J.; Chen, K.; Dharmarajan, K.; Adebola, S.; Danielczuk, M.; Mayoral-Vilches, V.; Jha, N.; Zhan, H.; Llontop, E.; Xu, D.; et al. FogROS2: An Adaptive Platform for Cloud and Fog Robotics Using ROS 2. In Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA), London, UK, 29 May–2 June 2023; pp. 5493–5500. [CrossRef]
79. Chen, K.; Hoque, R.; Dharmarajan, K.; LLontopl, E.; Adebola, S.; Ichnowski, J.; Kubiatowicz, J.; Goldberg, K. FogROS2-SGC: A ROS2 Cloud Robotics Platform for Secure Global Connectivity. In Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023; pp. 1–8.
80. Ismahil, D. Investigating Fog-and Cloud-Based Control Loops for Future Smart Factories. Bachelor's Thesis, Mälardalen University, Västerås, Sweden, 2017.
81. Lu, Y.; Yang, L.; Yang, S.X.; Hua, Q.; Sangaiah, A.K.; Guo, T.; Yu, K. An intelligent deterministic scheduling method for ultralow latency communication in edge enabled industrial Internet of Things. *IEEE Trans. Ind. Inform.* **2022**, *19*, 1756–1767. [CrossRef]
82. Benomar, Z.; Campobello, G.; Segreto, A.; Battaglia, F.; Longo, F.; Merlino, G.; Puliafito, A. A fog-based architecture for latency-sensitive monitoring applications in industrial internet of things. *IEEE Internet Things J.* **2021**, *10*, 1908–1918. [CrossRef]
83. Ji, C.H.; Hu, C.Y.; Liu, Z.D.; Ding, N. A Time-Sensitive Attribute-Oriented Multi-Message Dynamic Scheduling Method in ROS2-RTPS. In Proceedings of the 2023 IEEE International Conference on Smart Internet of Things (SmartIoT), Xining, China, 25–27 August 2023; pp. 211–217. [CrossRef]
84. RTI. The Request-Reply Pattern. 2025. Available online: https://bit.ly/3DrPRsB (accessed on 3 March 2025) .
85. Lienen, C.; Middeke, S.H.; Platzner, M. FPGADDS: An Intra-FPGA Data Distribution Service for ROS 2 Robotics Applications. In Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023; pp. 6261–6266. [CrossRef]
86. Zutell, J.M.; Conner, D.C.; Schillinger, P. ROS 2-Based Flexible Behavior Engine for Flexible Navigation. In Proceedings of the SoutheastCon 2022, Mobile, AL, USA, 26 March–3 April 2022; pp. 674–681. [CrossRef]
87. Kampmann, A.; Wüstenberg, A.; Alrifaee, B.; Kowalewski, S. A Portable Implementation of the Real-Time Publish-Subscribe Protocol for Microcontrollers in Distributed Robotic Applications. In Proceedings of the 2019 IEEE Intelligent Transportation Systems Conference (ITSC), Auckland, New Zealand, 27–30 October 2019; pp. 443–448. [CrossRef]
88. Poza-Luján, J.L.; Posadas-Yagüe, J.L.; Simo, J. Relationship between Quality of Control and Quality of Service in Mobile Robot Navigation. *Int. J. Imaging Robot.* **2014**, *12*, 90–105. [CrossRef]
89. Teper, H.; Betz, T.; Günzel, M.; Ebner, D.; Von Der Brüggen, G.; Betz, J.; Chen, J.J. End-To-End Timing Analysis and Optimization of Multi-Executor ROS 2 Systems. In Proceedings of the 2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS), Hong Kong, 13–16 May 2024; pp. 212–224. [CrossRef]

90. Fedi, F.; Nasca, F. Interoperability Issues Reduction in Command and Control for Multi-robot Systems. In *Modelling and Simulation for Autonomous Systems, Proceedings of the Second International Workshop, MESAS 2015, Prague, Czech Republic, 29–30 April 2015*; Hodicky, J., Ed.; Springer: Cham, Switzerland, 2015; pp. 77–89. [CrossRef]
91. Gamboa-Montero, J.J.; Alonso-Martin, F.; Marques-Villarroya, S.; Sequeira, J.; Salichs, M.A. Asynchronous federated learning system for human–robot touch interaction. *Expert Syst. Appl.* **2023**, *211*, 118510. [CrossRef]
92. Sundarapandian, S.; Kim, J.H.; Ferris-Francis, S.; Miller, J.; Prabakar, N.; Charters, C.; Michko, H. A Novel Communication Architecture and Control System for TeleBot: A Multi-Modal Telepresence Robot for Disabled Officers. *Int. J. Next Gener. Comput.* **2016**, *7*, 1–18. Available online: https://bit.ly/3StgL7j (accessed on 27 November 2024).
93. Criollo, L.; Mena-Arciniega, C.; Xing, S. Classification, military applications, and opportunities of unmanned aerial vehicles. *Aviation* **2024**, *28*, 115–127. [CrossRef]
94. Sandoval, S.; Thulasiraman, P. Cyber Security Assessment of the Robot Operating System 2 for Aerial Networks. In Proceedings of the 2019 IEEE International Systems Conference (SysCon), Orlando, FL, USA, 8–11 April 2019; pp. 1–8.
95. Phueakthong, P.; Varagul, J. A Development of Mobile Robot Based on ROS2 for Navigation Application. In Proceedings of the 2021 International Electronics Symposium (IES), Surabaya, Indonesia, 29–30 September 2021; pp. 517–520. [CrossRef]
96. Jalil, A.; Kobayashi, J. Experimental Analyses of an Efficient Aggregated Robot Processing with Cache-Control for Multi-Robot System. In Proceedings of the 2020 20th International Conference on Control, Automation and Systems (ICCAS), Busan-City, Republic of Korea, 13–16 October 2020; pp. 1105–1109. [CrossRef]
97. Yoo, T.; Choi, B.W. Interactive Path Editing and Simulation System for Motion Planning and Control of a Collaborative Robot. *Electronics* **2024**, *13*, 2857. [CrossRef]
98. Lourenço, L.L.; Oliveira, G.; Méa Plentz, P.D.; Röning, J. Achieving reliable communication between Kafka and ROS through bridge codes. In Proceedings of the 2021 20th International Conference on Advanced Robotics (ICAR), Virtual Event, 6–10 December 2021; pp. 324–329. [CrossRef]
99. Pradeep, V.; Abhi, B.C.; Abhilash, C.M.; Abhishek, M.S.; Adarsh. Robotics and Automation. *Int. J. Adv. Res. Sci. Commun. Technol. (IJARSCT)* **2024**, *4*, 129-134. [CrossRef]
100. Gutsche, K.; Genovese, J.; Serstjuk, P.; Altindis, S. User-centered design of professional social service robots. In Proceedings of the Intelligent Human Systems Integration (IHSI 2025): Integrating People and Intelligent Systems, Roma, Italy, 24–26 February 2025; Volume 160.
101. Hung, P.C.K.; Iqbal, F.; Aleem, S.; Rafferty, L. Snapshot of Research Issues in Service Robots. In *Service-Oriented Computing—ICSOC 2021 Workshops, Proceedings of the AIOps, STRAPS, AI-PA and Satellite Events, Dubai, United Arab Emirates, 22–25 November 2021*; Hacid, H.; Aldwairi, M.; Bouadjenek, M.R.; Petrocchi, M.; Faci, N.; Outay, F.; Beheshti, A.; Thamsen, L.; Dong, H., Eds.; Springer: Cham, Switzerland, 2022; pp. 363–376.
102. Jo, Y.H.; Cho, S.; Choi, B.W. Towards a ROS2-based software architecture for service robots. *Bull. Electr. Eng. Inform.* **2023**, *12*, 3027–3038. [CrossRef]
103. Cruz, J.M.; Romero-Garcés, A.; Rubio, J.P.B.; Robles, R.M.; Rubio, A.B. A DDS-based middleware for quality-of-service and high-performance networked robotics. *Concurr. Comput. Pract. Exp.* **2012**, *24*, 1761–1776. [CrossRef]
104. Darmanin, R.N.; Bugeja, M.K. A review on multi-robot systems categorised by application domain. In Proceedings of the 2017 25th Mediterranean Conference on Control and Automation (MED), Valletta, Malta, 3–6 July 2017; pp. 701–706. [CrossRef]
105. Deshpande, M.; Kamalanathan, N. Behavioral Analysis of ROS motion planners integrated with Robotics Middleware Framework (RMF). In Proceedings of the 2022 IEEE 4th International Conference on Cybernetics, Cognition and Machine Learning Applications (ICCCMLA), Goa, India, 8–9 October 2022; pp. 196–199. [CrossRef]
106. Fox, D.; Burgard, W.; Thrun, S. The dynamic window approach to collision avoidance. *IEEE Robot. Autom. Mag.* **1997**, *4*, 23–33. [CrossRef]
107. Rösmann, C.; Hoffmann, F.; Bertram, T. Integrated online trajectory planning and optimization in distinctive topologies. *Robot. Auton. Syst.* **2017**, *88*, 142–153. [CrossRef]
108. Macenski, S.; Singh, S.; Martín, F.; Ginés, J. Regulated pure pursuit for robot path tracking. *Auton. Robot.* **2023**, *47*, 685–694. [CrossRef]
109. Dey, E.; Walczak, M.; Anwar, M.S.; Roy, N.; Freeman, J.; Gregory, T.; Suri, N.; Busart, C. A Novel ROS2 QoS Policy-Enabled Synchronizing Middleware for Co-Simulation of Heterogeneous Multi-Robot Systems. In Proceedings of the 2023 32nd International Conference on Computer Communications and Networks (ICCCN), Honolulu, HI, USA, 24–26 July 2023; pp. 1–10. [CrossRef]
110. Aartsen, M.; Banga, K.; Talko, K.; Touw, D.; Wisman, B.; Meïnsma, D.; Björkqvist, M. Analyzing Interoperability and Security Overhead of ROS2 DDS Middleware. In Proceedings of the 2022 30th Mediterranean Conference on Control and Automation (MED), Athens, Greece, 28 June–1 July 2022; pp. 976–981. [CrossRef]
111. Open-Rmf. Open-RMF/free_fleet: A Free Fleet Management System. 2024. Available online: https://github.com/open-rmf/free_fleet (accessed on 5 March 2024).

112. Pramukantoro, E.S.; Anwari, H. An Event-based Middleware for Syntactical Interoperability in Internet of Things. *Int. J. Electr. Comput. Eng.* **2018**, *8*, 3784–3792. [CrossRef]
113. Jeong, S.; Ga, T.; Jeong, I.; Choi, J. Behavior Tree-Based Task Planning for Multiple Mobile Robots using a Data Distribution Service. In Proceedings of the 2022 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM), Hokkaido, Japan, 11–15 July 2022; pp. 1791–1798. [CrossRef]
114. Leung, K.Y.K.; Barfoot, T.D.; Liu, H.H.T. Distributed and decentralized cooperative simultaneous localization and mapping for dynamic and sparse robot networks. In Proceedings of the 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 9–13 May 2011; pp. 3841–3847. [CrossRef]
115. Kronauer, T.; Pohlmann, J.; Matthé, M.; Smejkal, T.; Fettweis, G. Latency Analysis of ROS2 Multi-Node Systems. In Proceedings of the 2021 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI), Karlsruhe, Germany, 23–25 September 2021; pp. 1–7. [CrossRef]
116. Essers, M.; Vaneker, T. Evaluating a Data Distribution Service System for Dynamic Manufacturing Environments: A Case Study. *Procedia Technol.* **2014**, *15*, 621–630.
117. Steve Cardenas, I.; Kumar Paladugula, P.; Kim, J.H. Large Scale Distributed Data Processing for a Network of Humanoid Telepresence Robots. In Proceedings of the 2020 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS), Vancouver, BC, Canada, 9–12 September 2020; pp. 1–9. [CrossRef]
118. Park, J.; Delgado, R.; Choi, B.W. Real-Time Characteristics of ROS 2.0 in Multiagent Robot Systems: An Empirical Study. *IEEE Access* **2020**, *8*, 154637–154651. [CrossRef]
119. Calisi, D.; Fedi, F.; Leo, A.; Nardi, D. Software Development for Networked Robot Systems. *IFAC Proc. Vol.* **2010**, *43*, 605–610.
120. Maruyama, Y.; Kato, S.; Azumi, T. Exploring the performance of ROS2. In Proceedings of the 2016 International Conference on Embedded Software (EMSOFT), Pittsburgh, PA, USA, 2–7 October 2016; pp. 1–10. [CrossRef]
121. Puck, L.; Keller, P.; Schnell, T.; Plasberg, C.; Tanev, A.; Heppner, G.; Roennau, A.; Dillmann, R. Performance Evaluation of Real-Time ROS2 Robotic Control in a Time-Synchronized Distributed Network. In Proceedings of the 2021 IEEE 17th International Conference on Automation Science and Engineering (CASE), Lyon, France, 23–27 August 2021; pp. 1670–1676. [CrossRef]
122. Deng, G.; Xu, G.; Zhou, Y.; Zhang, T.; Liu, Y. On the (In)Security of Secure ROS2. Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, Los Angeles, CA, USA, 7–11 November 2022.
123. Morita, R.; Matsubara, K. Dynamic Binding a Proper DDS Implementation for Optimizing Inter-Node Communication in ROS2. In Proceedings of the 2018 IEEE 24th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), Hakodate, Japan, 29–31 August 2018; pp. 246–247. [CrossRef]
124. Sanderson, D.; Shires, E.; Chaplin, J.C.; Brookes, H.; Liaqat, A.; Ratchev, S. Context-aware plug and produce for robotic aerospace assembly. In Proceedings of the Smart Technologies for Precision Assembly: 9th IFIP WG 5.5 International Precision Assembly Seminar, IPAS 2020, Virtual Event, 14–15 December 2020; Revised Selected Papers 9; Springer: Cham, Switzerland, 2021; pp.184–199.
125. Bovenzi, A.; Napolitano, A.; Esposito, C.; Carrozza, G. JFIT: An Automatic Tool for Assessing Robustness of DDS-Compliant Middleware. In *Innovative Technologies for Dependable OTS-Based Critical Systems: Challenges and Achievements of the CRITICAL STEP Project*; Cotroneo, D., Ed.; Springer: Milan, Italy, 2013; pp. 69–81. [CrossRef]
126. Laranjeiro, N.; Vieira, M.; Madeira, H. Experimental Robustness Evaluation of JMS Middleware. In Proceedings of the 2008 IEEE International Conference on Services Computing, Honolulu, HI, USA, 8–11 July 2008; Volume 1; pp. 119–126. [CrossRef]


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.